

Analyse de séquences appliquée aux trajectoires de vie : comparaison des bibliothèques disponibles sous R et Python et évaluation de l'apport des méthodes d'intelligence artificielle

RAPPORT DE STAGE - M2 STATISTIQUES ET SCIENCES DES DONNÉES

Garance MALNOË
encadrée par Caroline BAZZOLI (TIMC) et Jérôme GENSEL (LIG)

Université Grenoble Alpes

Mars - juillet 2026



Ce travail a bénéficié d'une aide de l'Etat gérée par l'Agence Nationale de la Recherche au titre du programme
«Investissements d'avenir » portant la référence ANR-22-CMAS-0005.

Table des matières

1	Introduction	1
2	Comparaison des environnements disponibles sous R et Python	4
2.1	Problématique	4
2.2	Méthodes et données	4
2.2.1	Protocole	4
2.2.2	Versions des bibliothèques utilisées	5
2.2.3	Données	5
2.3	Résultats	6
2.3.1	Formats des données	6
2.3.2	Données exemples et outils de simulation intégrés	7
2.3.3	Manipulation des données	8
2.3.4	Descriptions statistiques et visualisations	8
2.3.5	Construction de typologies	9
2.3.6	Comparaison des performances de temps de calcul	10
2.3.7	Autres types d'analyses de séquences	11
2.3.8	Outils de traitement des autres types de séquences	11
2.4	Discussion	13
3	Construction de typologies de séquences : comparaison entre méthodes classiques et deep clustering	14
3.1	Problématique	14
3.2	Méthodes et données	14
3.2.1	Méthodes comparées	14
3.2.2	Données simulées	16
3.2.3	Métriques d'évaluation	17
3.2.4	Application au Swiss Household Panel	17
3.3	Résultats	18
3.3.1	Impact des perturbations	18
3.3.2	Qualité du clustering	19
3.3.3	Application au Swiss Household Panel	19
3.4	Discussion	20
4	Utilisation de l'Intelligence Artificielle	21
5	Impact environnemental et social	21
5.1	Organisme d'accueil	21
5.2	Enjeux environnementaux personnels	22
5.3	Enjeux environnementaux globaux	22
5.4	Enjeux sociaux globaux	23
6	Discussion et conclusion	24
	Références	26
7	Annexes	28

1 Introduction

L’analyse de données longitudinales, et plus particulièrement l’analyse de séquences, vise à étudier l’organisation et l’évolution de trajectoires individuelles. Dans ce cadre, une séquence représente l’évolution d’une variable d’observation qualitative, le plus souvent selon une dimension temporelle. L’analyse de séquences trouve ainsi des applications dans de nombreux domaines de recherche en s’appuyant sur un large éventail de méthodes permettant notamment de visualiser et comparer les trajectoires, d’identifier des structures sous-jacentes ou encore de construire des typologies [1, 2]. À titre d’exemple, ces approches peuvent être utilisées pour étudier l’évolution de la structure familiale au sein d’un foyer ou les différents modes de transport utilisés par un individu au cours de ses déplacements domicile-travail.

Plus formellement, soient $I = [t_{min} ; t_{max}]$ l’horizon temporel pendant lequel nous nous intéressons aux variations d’une variable donnée et Σ , appelé *alphabet*, l’ensemble des états (ou modalités) que peut prendre cette variable¹, une séquence S est une liste de n triplets (t_i, t_{i+1}, σ) , appelés *épisodes*, avec t_i la date de début de l’épisode, t_{i+1} la date de fin de l’épisode et $\sigma \in \Sigma$ la valeur prise par la variable durant l’intervalle $[t_i ; t_{i+1}[$. Dans cette définition, les intervalles $[t_i ; t_{i+1}[$ doivent recouvrir l’ensemble de l’horizon temporel I sans se superposer. Toutefois, deux objets différents (par exemple deux individus ou deux foyers différents), n’ont pas nécessairement le même horizon temporel (*ie.* les séquences n’ont pas forcément la même longueur) [3].

Reprenons l’exemple de l’étude des mobilités pour le trajet domicile-travail. Nous pouvons alors définir l’alphabet $\Sigma_{déplacements} = \{\text{À pied, Vélo, Train, Bus, Voiture, Métro, Tram}\}$, où chaque élément représente un mode de transport susceptible d’être emprunté. Les séquences S_1 et S_2 suivantes sont deux exemples de trajets individuels :

$$S_1 = ((t_1, t_3, \text{À pied}), (t_3, t_4, \text{Métro}), (t_4, t_5, \text{À pied}))$$

$$S_2 = ((t_1, t_2, \text{Vélo}), (t_2, t_4, \text{Train}), (t_4, t_6, \text{Métro}), (t_6, t_7, \text{Vélo}))$$

Les horizons temporels respectifs de ces deux séquences sont $[t_1 ; t_5]$ et $[t_1 ; t_7]$. Cette représentation formelle capture simultanément l’ordre des modes de transport empruntés et la durée de chaque segment du trajet. La Figure 1 en propose une visualisation plus intuitive, sous la forme de chronogrammes colorés couramment utilisés dans la littérature [2].

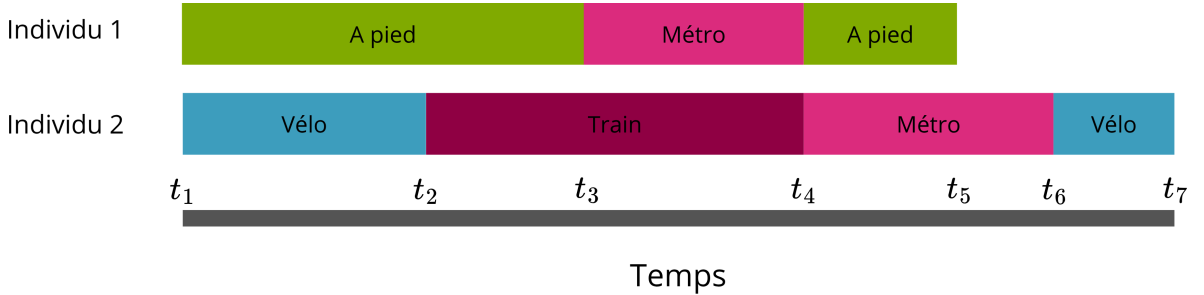


FIGURE 1 – Exemples de deux séquences décrivant les modes de transport utilisés pour le trajet domicile-travail

Il est possible d’étendre ce cadre initial à l’étude simultanée de plusieurs variables (ou attributs) pour un même objet d’observation (ex. une même individu, un même foyer). Ce qui amène alors à la notion de trajectoires multidimensionnelles (on parlera également de *multichannel*). Cette notion permet de mieux appréhender les notions de parcours de parcours de vie en mettant en parallèle, par exemple, le statut marital, l’activité professionnelle et le lieu de résidence. Toutefois, même si cette mise en parallèle de l’évolution de plusieurs attributs (ou variables) permet de mieux rendre compte de la complexité des trajectoires multidimensionnelles, cette vision ne capture pas nécessairement toutes les spécificités structurelles et sémantiques du véritable objet étudié. Le modèle **SalTo** [3, 4], pour *Semantic Life Trajectories*, propose une définition dans laquelle les trajectoires complexes sont segmentées en plusieurs thématiques regroupant un ensemble d’attributs (variables) proches sémantiquement. Au sein de la séquence associée à chaque variable, les états étudiés présentent une alternance entre des périodes

1. Par exemple $\Sigma = \{\text{célibataire, marié, divorcé, veuf}\}$ pour l’étude du statut marital.

de constance et des instants de transition. Dans le cadre du modèle SaLTo, ces deux notions correspondent respectivement à des *épisodes* et à des *événements*. Par conséquent, nous distinguerons les séquences d'*épisodes* (ci-après nommées séquences d'*états*), qui répondent à la définition précédente, et les séquences d'*événements*. Ces dernières peuvent être formellement définies comme une liste S_{ev} de couples $(t_i, description)$ où t_i désigne l'instant précis de la transition et $description$ constitue un champ libre permettant d'expliciter la variation de la valeur de l'attribut. La Figure 2 présente un exemple de cette formalisation avec une trajectoire de vie articulée autour de trois thématiques (familiale, résidentielle et professionnelle) composées chacune de deux à trois attributs.

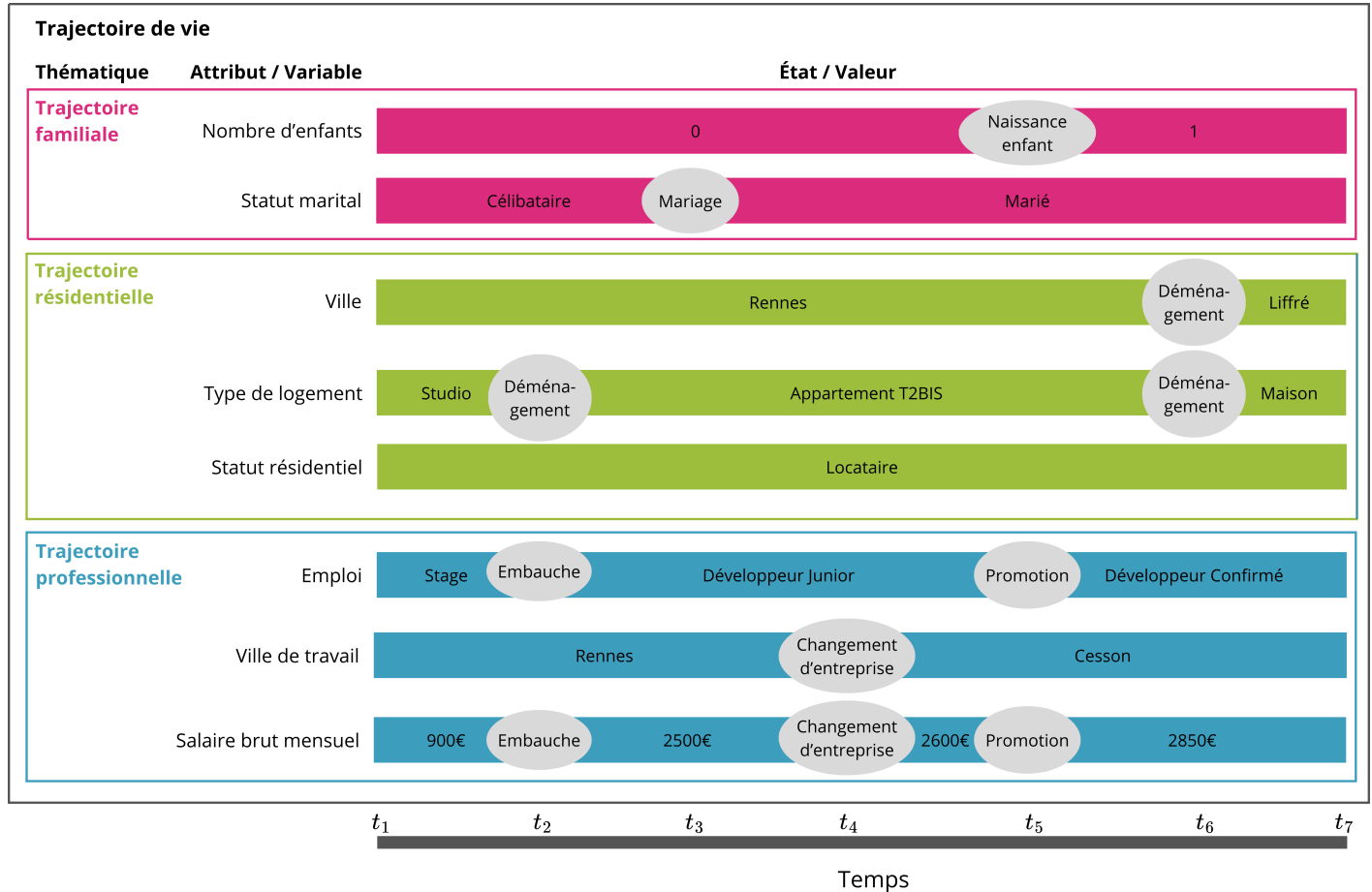


FIGURE 2 – Exemple de trajectoire de vie dans le modèle SaLTo. Les rectangles représentent les états et les cercles les événements associés aux transitions entre états.

De plus, il existe un troisième type de séquence proposant un compromis entre les séquences d'épisodes qui recouvrent l'ensemble de l'horizon temporel et les séquences d'événements qui ne correspondent qu'à un seul instant : des séquences d'*intervalles* ne recouvrant qu'une partie de l'horizon temporel et pouvant se chevaucher. Nous pouvons les définir similairement comme une liste S_{inter} de triplets (t_i, t_{i+1}, σ) , appelés *intervalles*, avec t_i la date de début de l'intervalle, t_{i+1} la date de fin de l'intervalle et $\sigma \in \Sigma$ la valeur prise par la variable considérée durant l'intervalle $[t_i; t_{i+1}[$, sachant que l'union des intervalles $[t_i; t_{i+1}[$ ne doit pas nécessairement recouvrir l'horizon temporel et que ces intervalles peuvent se chevaucher ou non.

La Figure 3 présente un exemple de trajectoire multidimensionnelle composée d'une séquence d'événements, d'une séquence d'intervalles et d'une séquence d'états.

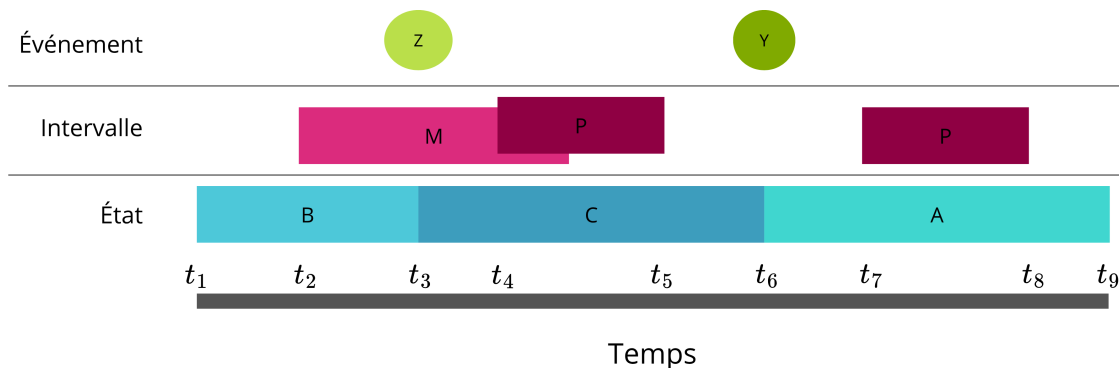


FIGURE 3 – Exemple de séquences d'états, d'intervalles et d'évènements

C'est donc dans ce contexte que s'est inscrit ce stage, réalisé au sein de deux laboratoires grenoblois : le laboratoire TIMC (Recherche Translationnelle et Innovation en Médecine et Complexité) et le Laboratoire d'Informatique de Grenoble (LIG). Ce stage s'est déroulé plus précisément au sein de l'équipe MESP (Modélisation et Évaluation des données complexes en Santé Publique) du laboratoire TIMC, dont les travaux portent notamment sur le développement de méthodes statistiques pour l'analyse de données médicales complexes et sur leur adaptation aux exigences de la recherche clinique^{2,3} et au sein de l'équipe STORIES (Semantic Trajectories / Spatio-Temporal trajectories) du LIG qui développe des méthodes de représentation, d'interrogation et d'analyse de trajectoires sémantiques afin de mieux décrire et comprendre l'évolution d'objets ou de phénomènes au cours du temps⁴. Depuis janvier 2026, ces deux équipes collaborent, aux côtés de l'INED⁵ et du laboratoire Espaces et Sociétés de l'université Rennes 2, dans le cadre du projet TRAVERSÉES qui vise à explorer de nouvelles possibilités d'analyse des transformations sociales et démographiques à partir de données individuelles issues d'enquêtes biographiques. Ce stage s'inscrit plus précisément dans le lot 5 du projet, consacré à l'exploration statistique, et plus particulièrement à l'évaluation et l'adaptation de méthodes d'analyse de séquences déjà existantes aux spécificités des objets du projet, aux possibilités d'interprétations collectives à partir de données de parcours individuels, tout en utilisant des données simulées et réelles pour mener ces évaluations⁶.

Ce stage s'est ainsi structuré autour de l'analyse de séquences, abordée selon deux niveaux d'étude. Le premier a porté sur les outils logiciels permettant de conduire ces analyses : il a consisté à comparer les principales bibliothèques disponibles sous R et Python, tant au regard des méthodes implémentées que de leur capacité à prendre en charge des trajectoires complexes et multidimensionnelles, notamment celles décrites par le modèle SaLTo et les objets du projet TRAVERSÉES. Le second s'est concentré sur une tâche spécifique de l'analyse de séquences, la construction de typologies, qui vise à regrouper des trajectoires similaires afin d'identifier des profils types. Alors que les approches classiques reposent généralement sur le calcul préalable de dissimilarités entre trajectoires puis sur leur classification, ce second axe a étudié dans quelle mesure des méthodes issues de l'intelligence artificielle (IA), et plus particulièrement du deep clustering, peuvent constituer une alternative en apprenant directement une représentation latente des trajectoires au travers d'architectures d'auto-encodeurs.

La suite de ce rapport présente successivement ces deux volets. La première partie est consacrée à la comparaison des environnements logiciels disponibles sous R et Python pour l'analyse de séquences. La deuxième partie porte sur l'étude de méthodes de deep clustering appliquées à la construction de typologies de trajectoires multidimensionnelles et sur leur comparaison avec des approches classiques. Enfin, les deux dernières parties reviennent respectivement sur l'usage des outils d'IA au cours du stage et sur les enjeux environnementaux et sociétaux du projet à différentes échelles.

2. <https://www.timc.fr/laboratoire>

3. <https://www.timc.fr/axes-recherche/equipes-recherche/mesp>

4. <https://www.liglab.fr/fr/recherche/equipes-recherche/stories-semantic-trajectories-spatio-temporal-trajectories>

5. Institut National d'Études Démographiques

6. <https://lig-traversees-anr.imag.fr/>

2 Comparaison des environnements disponibles sous R et Python

2.1 Problématique

Jusqu'à présent, le langage R s'impose comme le langage de référence pour l'implémentation des méthodes d'analyse de séquences, notamment au travers du package **TraMineR**, mis à disposition depuis 2008, et de son extension **TraMineRextras**⁷ [5]. Ceux-ci permettent d'importer et de mettre en forme des données longitudinales sous la forme d'objets séquences, qui peuvent ensuite être manipulés pour mettre en œuvre différentes méthodes d'analyse proposées par ces packages. Bénéficiant de près de deux décennies de développement, d'une communauté d'utilisateurs active et d'une large utilisation dans la littérature, **TraMineR** constitue aujourd'hui une référence dans le domaine de l'analyse de séquences. Dix autres packages référencés par la *Sequence Analysis Association*⁸ complètent cet écosystème (chaînes de Markov, visualisations au format **ggplot2**, ...). Toutefois, si R est historiquement conçu pour l'analyse statistique et bénéficie d'un écosystème particulièrement mature, Python est un langage plus généraliste qui occupe aujourd'hui une place importante et toujours croissante en sciences des données [6]. Le développement d'outils d'analyse de séquences dans cet environnement apparaît dès lors pertinent, notamment pour bénéficier de son plus large écosystème de bibliothèques et explorer de nouvelles possibilités d'implémentation. C'est dans cette perspective que s'inscrivent **TanaT**⁹, mis à disposition en 2024, et **sequenzo**¹⁰, mis à disposition en 2025. De plus, au-delà de reproduire les fonctionnalités de **TraMineR**, **TanaT** et **sequenzo** ambitionnent de les étendre : d'une part, **sequenzo** propose de traiter des données massives ; d'autre part, **TanaT** vise à prendre en charge des trajectoires mixtes composées de séquences de différentes natures : des séquences d'états, des séquences d'évènements, mais également des séquences d'intervalles [7] ainsi que des séquences de données quantitatives, ce qui pourrait se montrer particulièrement intéressant pour l'implémentation du modèle **SalTo**.

L'objectif de la première partie de ce stage a donc été d'explorer en détail les possibilités qu'offrent les bibliothèques **TanaT** et **sequenzo** et de les confronter à celles de l'écosystème existant de **TraMineR**. Il s'agit plus précisément de : **1)** déterminer si ces nouvelles bibliothèques permettent en effet de réaliser les mêmes tâches que celles déjà couvertes par **TraMineR** ; **2)** avec quel gain potentiel de "performances" pour la construction de typologies ; **3)** de faire un inventaire des nouvelles implémentations, notamment pour les séquences d'intervalles et d'évènements pour **TanaT** ; et **4)** d'évaluer la capacité de **TanaT**, **sequenzo** et **TraMineR** à traiter des données au format du modèle **SalTo**.

2.2 Méthodes et données

2.2.1 Protocole

Pour répondre à ces problématiques, une comparaison systématique des fonctionnalités de **TraMineR** (et **TraMineRextras**), **TanaT** et **sequenzo** a été menée, en considérant à la fois les fonctionnalités natives et celles accessibles via des bibliothèques tierces recommandées dans leur documentation. Sept axes d'évaluation ont été retenus : **1)** les formats de données pris en charge, **2)** les jeux de données exemples et outils de simulation, **3)** la gestion et le filtrage des séquences, **4)** les descriptions statistiques et visualisations, **5)** la construction de typologies (distances, clustering et analyses multidomaines), ainsi que les performances associées, **6)** les autres analyses disponibles (survie, risques concurrents, SHA, ...) ¹¹, et **7)** la prise en charge des séquences d'intervalles et d'évènements.

Lorsque certaines opérations n'étaient prévues, ni nativement par **TraMineR**, **TanaT** ou **sequenzo**, ni par l'intermédiaire de bibliothèques tierces explicitement prévues dans leur documentation ou leurs exemples d'utilisation, des solutions alternatives ont, dans la très grande majorité des cas, été réimplémentées à partir d'outils standards

7. <https://traminer.unige.ch/>

8. <https://sequenceanalysis.org/software/>

9. <https://tanat.gitlabpages.inria.fr/core/tanat/index.html>

10. <https://sequenzo.yuqi-liang.tech/en/basics/about-sequenzo>

11. **survie** : étude de la durée écoulée avant la survenue d'un événement ; **risques concurrents** : étude de la survenue d'un événement et de la trajectoire qui s'ensuit sur une période donnée ; **SHA** (*Sequence History Analysis*) : étude de l'effet d'une trajectoire passée sur la survenue d'un événement.

de manipulation de données, tels que `dplyr` sous R ou `pandas` et `numpy` sous Python, afin de pouvoir permettre une comparaison plus complète et de proposer une référence pour l’implémentation de chaque tâche pour chacune des trois librairies. Seules deux visualisations (*single medoid plot* et *relative frequency plot*), identifiées lors de l’étude de `sequenzo`, menée dans un second temps par rapport à celles de `TraMineR` et `TanaT`, n’ont pas fait l’objet d’une réimplémentation rétrospective pour les deux autres librairies. Ces différents cas seront signalés dans la suite du document.

Les résultats présentés par la suite proposent une synthèse des principales différences entre `TraMineR`, `TanaT` et `sequenzo`, ainsi que des fonctionnalités disponibles ou absentes dans chacune de ces librairies, sans détailler les implémentations ni les sorties obtenues. C’est pourquoi, en complément, un document PDF regroupant le code utilisé et les sorties associées à chacune des tâches du protocole a été produit pour chaque librairie au cours du stage. Ces documents constituent ainsi à la fois un support détaillé pour la comparaison menée et une référence d’implémentation pour l’utilisation de chaque librairie.

2.2.2 Versions des librairies utilisées

Pour réaliser cette analyse, R (4.5.2) a été utilisé avec `TraMineR` (2.2-13), `TraMineRextras` (0.6.8), `cluster` (2.1.8.1), `dplyr` (1.2.0), `coxme` (2.2-22), `survival` (3.8-3), `nnet` (7.3-20) et `mlogit` (1.1-3). La librairie `sequenzo` n’étant, au moment de l’analyse, compatible qu’avec les versions 3.9 à 3.11 de Python, deux environnements Python distincts ont été utilisés : `sequenzo` avec Python (3.11.9), `sequenzo` (0.1.38), `matplotlib` (3.10.8), `numpy` (2.4.4), `scipy` (1.16.3) et `pandas` (3.0.2) ; et `TanaT` avec Python (3.14.3), `TanaT` (0.9.0), `matplotlib` (3.10.8), `numpy` (2.4.3), `scipy` (1.17.1) et `pandas` (3.0.1). Les évaluations de performances ont été réalisées sur la même machine, équipée de 16,0 Go de RAM et d’un processeur Intel i5-1335U (1,30 GHz).

2.2.3 Données

Pour réaliser cette comparaison systématique, les mêmes jeux de données ont été utilisés pour les trois environnements. Pour cela, l’analyse s’est appuyée sur deux jeux de données issus du *Swiss Household Panel* (SHP) ¹² : une étude longitudinale d’envergure nationale lancée en 1999 recueillant annuellement des informations sociodémographiques et économiques auprès d’un échantillon représentatif de ménages suisses [8]. Pour les analyses de séquences unidimensionnelles, la comparaison s’est appuyée sur le jeu de données *Family life states* (`biofam`) ¹³. Ce choix a été motivé par son utilisation comme jeu de données d’exemple dans `TraMineR` et `sequenzo`, facilitant ainsi la comparaison des fonctionnalités proposées par ces deux librairies. Pour les analyses nécessitant des séquences multidimensionnelles portant sur plusieurs thématiques, aucun jeu de données existant suffisamment complet n’ayant été identifié, un extrait plus large du *Swiss Household Panel* (`extract_shp`) a été constitué spécifiquement pour cette étude.

Family life states (`biofam`) : le jeu de données *Family life states* (`biofam`) est issu de l’enquête biographique rétrospective menée en 2002 dans le cadre du SHP. Il documente les trajectoires unidimensionnelles de formation familiale d’un échantillon de 2 000 individus sur une période de seize années, sans données manquantes, de l’âge de 15 ans à l’âge de 30 ans. Ces séquences sont initialement structurées au format **STS** ¹⁴ et sont directement exploitables par `TraMineR` et `sequenzo`. Une version convertie au format **SPELL** ¹⁵ est utilisée pour `TanaT`. L’alphabet unique des séquences décrites dans ce jeu de données est composé de huit statuts familiaux décrivant simultanément le lieu de résidence, le statut marital et la parentalité, décrivant ainsi plusieurs thématiques avec une unique variable. Les individus sont ainsi catégorisés à chaque pas de temps selon qu’ils vivent au domicile parental (*Parent*), qu’ils l’ont quitté (*Left*), qu’ils ont un enfant (*Child*), qu’ils sont mariés (*Married*) qu’ils ont quitté le domicile familial et qu’ils sont mariés (*Left+Marr*), qu’ils ont quitté le domicile familial et ont un enfant (*Left+Child*), qu’ils ont quitté le domicile familial, mariés et avec un enfant (*Left+Marr+Child*) ou qu’ils soient divorcés (*Divorced*). Ainsi l’alphabet complet est

12. <https://forscenter.ch/projects/swiss-household-panel/>

13. <https://traminer.unige.ch/doc/biofam.html>

14. **STS** : Une ligne par objet et une colonne par point dans le temps.

15. **SPELL** : une colonne d’identifiant, une date de début, une date de fin et une colonne d’état.

$\Sigma = (Parent, Left, Child, Married, Left+Marr, Left+Child, Left+Marr+Child, Divorced)$.

Le jeu de données inclut également plusieurs covariables fixes transversales¹⁶, portant sur les caractéristiques socio-démographiques (sexe, année de naissance, première nationalité, langue de réponse), religieuses (religion et participation religieuse) et l'origine sociale (statut social des parents). Certaines comportant des valeurs manquantes, seule la variable sexe, complète, a été retenue pour les comparaisons. Deux jeux de poids¹⁷ sont également fournis, l'un ajusté sur la population suisse et l'autre ramené à la taille de l'échantillon. Ils n'ont toutefois pas été utilisés, le jeu de données constituant un sous-échantillon du panel pour lequel ces pondérations ne sont plus adaptées.

Extrait du Swiss Household Panel `extract_shp` : le jeu de données `extract_shp` est un extrait des réponses de l'enquête longitudinale du panel de ménages suisses entre 1999 et 2018 inclus. Seuls les ménages sans données manquantes et ayant participé à l'enquête chaque année ont été sélectionnés, constituant ainsi 20 observations (20 pas de temps réguliers) de 1032 foyers (objet d'étude). Trois thématiques ont été constituées, chacune regroupant une à trois variables (ou attributs) évoluant au fil du temps, pour un total de six variables.

1) Thématique familiale : pour cette thématique, seul le **nombre de personnes composant le ménage** est considéré. Cette variable prend des valeurs allant de 1 à 12, que nous regroupons en cinq catégories : 1, 2, 3, 4, et 5 habitants ou plus au sein du foyer.

2) Thématique résidentielle : cette thématique regroupe trois variables : la région d'habitation, le type de commune d'habitation et le statut résidentiel. La **région d'habitation** comporte sept modalités (ou états), les acronymes des cantons correspondants sont indiqués entre parenthèses : Lac Léman (VD, VS, GE), Middleland (BE, FR, SO, NE, JU), Suisse Nord-Ouest (BS, BL, AG), Zurich, Suisse Est (GL, SH, AR, AI, SG, GR, TG), Suisse Centrale (LU, UR, SZ, OW, NW, ZG) et Tessin. Le **type de commune d'habitation** comporte quant à lui neuf modalités : centres, communes de banlieue, communes aisées, communes urbaines périphériques, communes touristiques, communes industrielles et tertiaires, communes rurales de banlieue, communes mixtes agricoles et communes agricoles périphériques. Le **statut résidentiel** se décline enfin en trois modalités : locataire, propriétaire ou copropriétaire, et logé à titre gratuit.

3) Thématique qualité de vie : cette thématique comprend deux variables : l'état du lieu de vie du foyer et la satisfaction à l'égard de la situation financière du foyer. L'**état du lieu de vie du foyer** comporte trois modalités : en mauvais état, en bon état mais n'ayant pas été rénové récemment, et neuf ou récemment rénové. La **satisfaction à l'égard de la situation financière du foyer** est à l'origine mesurée sur une échelle entière de 0 (pas du tout satisfait) à 10 (totalement satisfait), et est regroupée en cinq catégories : 0-1, 2-3, 4-6, 7-8 et 9-10.

Cette base de données est initialement seulement disponible au format **LONG**¹⁸, qui n'est directement compatible avec aucun des trois packages. Il a donc fallu la transformer au format **STS** pour `TraMineR` et `sequenZo` et au format **SPELL** pour `TanaT`.

2.3 Résultats

2.3.1 Formats des données

Le premier volet de l'évaluation a porté sur la diversité des formats de données admis pour la construction d'objets séquences, étape préalable indispensable à toute analyse de séquences. L'examen réalisé, synthétisé dans la Table 1 en annexe, met en évidence des différences majeures entre les trois environnements.

L'écosystème `TraMineR` se montre très souple vis-à-vis des formats acceptés. Bien que les formats pris en charge pour la création d'un objet de séquences d'états se limitent au nombre de trois, `TraMineR` accepte une grande variété de formats de données brutes et fournit de nombreuses fonctions de conversion permettant de passer aisément d'un format à un autre. De plus, il intègre nativement la gestion des poids statistiques de redressement, permettant de pondérer le calcul d'indicateurs (fréquences d'états, entropie, etc.) afin de mieux refléter la

16. Variables dont la valeur ne change pas durant toute l'étude (ex. date de naissance, sexe attribué à la naissance).

17. Valeurs numériques permettant de redresser les calculs en pondérant les observations selon leur importance dans la population étudiée.

18. **LONG** : une ligne par individu par point de temps.

population source, ce qui peut se révéler particulièrement utile dans le cadre d’analyses sociologiques.

À l’inverse, **TanaT** est nettement plus restrictif. La bibliothèque n’accepte qu’un format **SPELL** allégé, composé d’une colonne d’identification de l’objet étudié, d’une colonne de date de début de l’épisode et d’une colonne décrivant l’état de l’épisode, sans colonne de date de fin de l’épisode, celle-ci étant calculée automatiquement. De plus, **TanaT** ne propose pas de fonctions de conversion depuis d’autres formats. Ainsi, lorsque les données ne suivent pas déjà le format **SPELL**, l’utilisateur doit réaliser lui-même une conversion préalable des données, en s’appuyant par exemple sur des bibliothèques de manipulation de données standards telles que **pandas**.

Le package **sequenzo** adopte une approche intermédiaire. Il accepte deux formats de données et propose également des fonctions permettant de convertir les données d’un format à l’autre. Il se montre donc plus restrictif que **TraMineR**, tout en restant plus flexible que **TanaT**. Comme **TraMineR**, il permet par ailleurs la prise en compte des poids statistiques dans les analyses.

2.3.2 Données exemples et outils de simulation intégrés

Le deuxième axe évaluait les jeux de données et les outils de simulation mis à disposition par les trois bibliothèques.

Jeux de données intégrés : les trois bibliothèques mettent à disposition des jeux de données variés, mais avec des profils sensiblement différents. **TraMineR** propose principalement des séquences d’états issues des sciences sociales, souvent accompagnées de nombreuses covariables fixes. Certains jeux ont également une vocation pédagogique, notamment pour illustrer la gestion des valeurs manquantes ou l’utilisation de poids de redressement. **TanaT** se distingue davantage par la diversité des types de séquences représentés, avec des données d’états, d’évènements et d’intervalles. Enfin, **sequenzo** propose le plus grand nombre de jeux de données, couvrant des thématiques variées.

Des différences apparaissent également dans la structure de ces données. Contrairement à **TraMineR** et **TanaT**, **sequenzo** fournit nativement plusieurs jeux de données multidimensionnels (*multichannel*) : **dyadic**, **political science** et **pairfam** comportent deux dimensions séquentielles, tandis que les données sur les pays en comportent quatre, bien que toutes les séquences n’aient pas nécessairement la même longueur. Les jeux de données de **TraMineR** et **TanaT** se limitent quant à eux à une seule variable séquentielle par individu. **TraMineR** se distingue toutefois par la richesse des covariables associées à ses séquences d’états, alors que **TanaT** n’en propose aucune, y compris pour le jeu de données commun **mvad**. À l’inverse, ce dernier fournit des exemples de séquences d’intervalles, absents de **TraMineR** et de **sequenzo**.

Ces caractéristiques sont synthétisées dans les Tables 2, 3 et 4 en annexe. Pour chaque jeu de données, celles-ci indiquent sa thématique, le type et le format des séquences, le nombre d’objets étudiés (Nb. obj)¹⁹, leur longueur (Long.), ainsi que le nombre d’états distincts (Nb. états) et de covariables fixes associées (Nb. cov.).

Simulations de données : concernant la génération de données synthétiques, **TanaT**, **TraMineR** et **sequenzo** proposent des approches et des niveaux de paramétrage différents.

L’écosystème **TraMineR** repose sur la fonction **seqgen()**, qui permet de générer n séquences d’états de longueur et d’alphabet fixés par l’utilisateur. Son principal atout réside dans la possibilité de spécifier les probabilités d’occurrence de chaque élément de l’alphabet. Un exemple est donné dans le Listing 1 en annexe.

De son côté, **TanaT** offre un spectre plus large de types de séquences générables : séquences d’états, d’évènements et d’intervalles, via les fonctions **generate_X_sequences()**. Sa particularité est d’intégrer nativement la simulation de données manquantes (remplacement des données par **NA** aléatoirement suivant une proportion fournie par l’utilisateur). En revanche, il ne permet pas de contrôler les probabilités d’occurrence des différents états, ils sont obligatoirement tirés selon une distribution uniforme. Un exemple est donné dans le Listing 2 en annexe.

Le package **sequenzo** est le seul à ne pas proposer de fonctionnalité de génération de données. Il est néanmoins possible de définir assez aisément, à l’aide des librairies **numpy** et **pandas**, une fonction regroupant les fonctionnalités de **TraMineR** et **TanaT**. Le Listing 3 en annexe présente un exemple d’implémentation reprenant

19. Correspond le plus souvent au nombre d’individus ou de foyers étudiés.

le mécanisme de données manquantes de **TanaT** et la génération aléatoire selon une distribution fournie par l'utilisateur de **TraMineR**.

2.3.3 Manipulation des données

Concernant la manipulation des séquences, **TanaT** se distingue par sa gestion native des filtres. La bibliothèque propose une méthode unifiée (`filter()`) permettant d'opérer sur cinq dimensions :

- **Query** : filtrage selon la présence ou l'absence d'états spécifiques dans la trajectoire (ex. séquences incluant l'état A) ;
- **Static** : filtrage selon les attributs ou covariables associés aux individus (ex. séquences correspondant uniquement aux femmes) ;
- **Time** : restriction à une période d'observation délimitée (ex. trajectoires observées uniquement entre 2001 et 2005) ;
- **Pattern** : filtrage selon un schéma de transitions donné (ex. passage de l'état A à B, puis à D) ;
- **Length** : filtrage selon la longueur totale de la séquence (ex. trajectoires de longueur strictement supérieure à 5).

Du côté de **TraMineR** et **sequenZo**, il n'existe pas de fonction native réalisant ces opérations de filtrage directement sur l'objet séquence. Toutefois, il est tout à fait possible de réaliser ces filtres par le biais des fonctions de base de R ou via des packages dédiés à la gestion des données comme **dplyr** pour **TraMineR** et par le biais de la librairie **pandas** pour **sequenZo**. Un exemple d'implémentation d'un filtrage pour chacun des cinq aspects sur le jeu de données **biofam** a été mis à disposition dans les documents PDF dédiés à la présentation de chacun des packages.

Enfin, **TraMineR** permet la modification de la granularité temporelle : le changement de pas de temps d'observation des séquences (ex.: passage d'une séquence observée tous les mois à tous les ans en prenant l'état le plus fréquent durant les douze mois de l'année). Cette fonctionnalité peut s'avérer particulièrement utile pour un utilisateur travaillant sur des trajectoires composées de variables observées à des pas de temps différents ou si tous les individus n'ont pas répondu avec le même pas de temps. **TraMineR** intègre des fonctions dédiées permettant de gérer cet aspect tandis que **TanaT** et **sequenZo** ne proposent aucune méthode spécifique pour cette tâche, mais qui peut toutefois être réalisée en modifiant au préalable les données initiales avec la bibliothèque **pandas**. Un exemple sur le jeu de données **biofam** a été proposé dans le document spécifique à chaque librairie.

2.3.4 Descriptions statistiques et visualisations

Concernant la visualisation des séquences ainsi que les descriptions statistiques du jeu de données et leurs représentations graphiques associées, les trois écosystèmes présentent des niveaux de complétude sensiblement différents, comme le synthétise la Table 5 en annexe. **TraMineR** se distingue comme l'environnement le plus complet sur ce plan : il propose quasi systématiquement une sortie en données brutes (liste ou tableau de statistiques) associée à une visualisation. La librairie **sequenZo** offre une couverture presque équivalente à celle de **TraMineR** en matière de visualisations, tout en proposant deux représentations supplémentaires : le *single medoid plot*²⁰ et le *relative frequency plot*²¹. Il est toutefois à noter que la première de ces fonctionnalités est bien décrite dans la documentation de la librairie, mais n'a toutefois pas pu être exécutée avec succès dans l'environnement utilisé pour ce comparatif. En revanche, **sequenZo** ne permet généralement pas de récupérer les statistiques sous-jacentes aux visualisations sous forme de valeurs brutes, à l'exception de l'entropie *transversale*²². **TanaT**

20. Représente la séquence la plus centrale (médoïde) pour plusieurs groupes selon une matrice de distance fournie par l'utilisateur.

21. Produit un graphique à deux volets résumant la "typicité" des séquences au sein de groupes de fréquence. Le premier représente la séquence médiane de chaque groupe, tandis que le second présente, sous la forme de diagrammes en boîte, les dissemblances entre les séquences et le médoïde de leur groupe. Une ligne de synthèse fournit également un pseudo- R^2 , une statistique F et une p-valeur caractérisant la séparation entre les groupes.

22. L'entropie *transversale* mesure la diversité des états observés parmi l'ensemble des séquences à chaque point de temps. Elle est définie, pour un point de temps donné, par $h(p_1, \dots, p_s) = -\sum_{i=1}^s p_i \log(p_i)$, où s désigne la taille de l'alphabet et $p_1, \dots, p_s$ les fréquences respectives des différents états. Elle est minimale lorsque toutes les séquences se trouvent dans le même état et maximale lorsque les états sont représentés dans les mêmes proportions.

apparaît être le moins exhaustif des trois : il ne propose que trois visualisations (la représentation des séquences brutes sous forme de chronogramme, un barplot pour la fréquence des états et un graphique des aires empilées pour la distribution transversale) et ne propose aucune sortie statistique brute. Il convient toutefois de noter que, pour un utilisateur à l’aise avec `pandas`, `scipy` et `numpy`, il est relativement aisé de calculer ces statistiques et d’obtenir des visualisations grâce à `matplotlib`. Pour l’ensemble des sorties (brutes ou visuelles) non proposées par les librairies, à l’exception du *single medoid plot* et du *relative frequency plot* comme présenté dans le protocole, un exemple d’implémentation a été proposé dans le document de chaque librairie correspondante. À titre d’exemple, les listings 4 et 5 en annexe illustrent une possible implémentation du calcul des fréquences de séquences pour **TanaT** et **sequenzo** respectivement.

En ce qui concerne la gestion des visualisations selon une variable (ex. un groupement d’une covariable fixe ou un partitionnement issu d’un clustering), **TraMineR** et **sequenzo** prennent en charge nativement la production de figures subdivisées en sous-graphiques correspondant à chaque modalité. En revanche, **TanaT** ne dispose pas de cette fonctionnalité : l’utilisateur doit procéder à un filtrage manuel préalable des données afin de générer une figure distincte pour chaque groupe. Un exemple d’implémentation d’un chronogramme pour les séquences du jeu de données **biofam**, séparées selon la variable catégorielle **sex**, est fourni pour chacun des trois environnements dans les listings 6, 7 et 8 en annexe. Les Figures 10, 11 et 12 en annexe présentent les sorties correspondantes. Sur le plan des statistiques descriptives des séquences individuelles, les trois bibliothèques présentent des couvertures partiellement complémentaires (Table 6 en annexe). L’entropie *within*²³ constitue le seul indicateur commun aux trois bibliothèques, **TraMineR** et **sequenzo** sont les seules à proposer des mesures de complexité structurelle (turbulence, complexité) [1] mais **TanaT** est celle qui propose le plus d’indicateurs sur les durées des épisodes. **sequenzo** est le seul environnement à proposer une visualisation dédiée à ces statistiques : un diagramme en barres regroupant le nombre de transitions, l’entropie *within*, la turbulence et la complexité. Ainsi, les trois bibliothèques présentent des couvertures partiellement complémentaires : chacune calcule nativement certains indicateurs absents des deux autres, sans qu’aucune ne s’impose comme une référence exhaustive.

En matière de personnalisation graphique, les trois écosystèmes sont comparables : tous permettent l’édition de la palette de couleurs ainsi que la modification de paramètres graphiques généraux (titres, taille des axes, etc.). Pour l’écosystème de **TraMineR**, il est possible de passer dans l’environnement visuel de `ggplot2` avec le package `ggseqplot` pour les utilisateurs qui le souhaiteraient.

2.3.5 Construction de typologies

La construction de typologies occupe une place centrale en analyse de séquences : il s’agit de regrouper les séquences similaires en profils types, dont les caractéristiques peuvent ensuite être interprétées à la lumière de covariables et de l’expertise métier propre au domaine d’étude. La construction de ces typologies repose sur deux étapes successives : le calcul d’une mesure de dissimilarité pour chaque paire de séquences, aboutissant à une matrice de distances entre toutes les séquences, puis l’application d’un algorithme de classification (clustering) à cette matrice afin de constituer des groupes (idéalement) homogènes. Pour chacune de ces étapes, **TraMineR**, **sequenzo** et **TanaT** offrent des possibilités différentes.

Comparaison des possibilités de métriques de dissimilarité : la Table 7 en annexe synthétise les possibilités de métriques de dissimilarité entre séquences qu’offrent les trois librairies. Bien que **sequenzo** propose nativement un plus grand nombre de distances que **TraMineR** et **TanaT**, cette dernière bibliothèque se distingue par le degré de liberté qu’elle offre à l’utilisateur. D’une part, elle permet d’implémenter des fonctions de distance personnalisées, tant entre entités (la distance de Hamming étant utilisée par défaut) qu’entre séquences, conférant ainsi une flexibilité absente des deux autres environnements. D’autre part, **TanaT** est la seule à pouvoir traiter des variables continues, et non exclusivement catégorielles, dès lors que l’utilisateur spécifie une fonction de distance entre entités appropriée.

23. L’entropie *within* correspond à l’entropie de chaque séquence individuelle. Elle peut être interprétée comme une forme d’incertitude dans la prédiction des états de la séquence. Elle est définie comme $h(\pi_1, \dots, \pi_s) = -\sum_{i=1}^s \pi_i \log(\pi_i)$ où s est la taille de l’alphabet, $\pi_1, \dots, \pi_s$ sont les fréquences de chaque état dans la séquence considérée. Par défaut dans **TraMineR**, la version normalisée (entropie de la séquence divisée par l’entropie de l’alphabet) est utilisée.

Comparaison des possibilités d’algorithmes de clustering : pour l’étape de clustering, les trois bibliothèques adoptent des approches différentes. **TraMineR** prévoit nativement de faire appel à des packages tiers pour réaliser cette tâche (notamment le package **cluster** dans sa documentation [5]), ce qui permet de recourir à tout algorithme déjà implémenté ayant recours uniquement à une matrice de dissimilarité. **TanaT** et **sequenzo** privilégient en revanche une approche intégrée [7]. Les deux bibliothèques Python proposent nativement le groupement hiérarchique agglomératif (AGNES) ainsi que deux algorithmes de *K-medoids* : *Partitioning Around Medoids* (PAM) et *Clustering Large Applications* (CLARA). Cette intégration facilite la manipulation directe des objets de séquences au sein de la bibliothèque, mais limite le choix des algorithmes accessibles nativement dans le flux de travail, par exemple pour recourir au *fuzzy clustering* (disponible avec le package **cluster** en R) ou à une implémentation personnalisée. D’autres algorithmes restent néanmoins utilisables : l’utilisateur peut extraire la matrice de dissimilarité calculée par **TanaT** ou **sequenzo**, l’utiliser comme entrée d’un algorithme externe, puis rattacher les groupes obtenus aux séquences. Cette procédure nécessite toutefois de sortir de l’interface intégrée de la bibliothèque et de manipuler séparément la matrice de distances et les résultats du clustering, contrairement aux algorithmes directement pris en charge.

Cas multidimensionnel (*multichannel*) : l’analyse de séquences ne se limite pas qu’au cas unidimensionnel et peut également être intéressante à appliquer dans un contexte multidimensionnel comme celui du modèle **SaLTo** présenté en introduction. Le processus analytique dans le cadre multidimensionnel reste sensiblement le même : il s’agit toujours de définir des séquences à partir d’un certain alphabet d’états, de choisir le coût du passage d’un état à un autre, de calculer la dissimilarité entre les séquences et d’appliquer un algorithme de clustering pour obtenir des groupements de séquences similaires. L’enjeu devient alors de choisir le moment auquel considérer l’aspect multidimensionnel. Quatre approches sont distinguées dans la littérature [9], schématisées dans la Figure 13 en annexe, : l’extension de l’alphabet (EA, *Extended Alphabet*), l’association de matrices de coût (CAT, *Cost Additive Trick*), l’association des matrices de dissimilarité (DAT, *Distance Additive Trick*) et enfin la combinaison des clusters unidimensionnels (CombT, *Combining Trajectories*).

Au sein de l’écosystème **TraMineR**, la stratégie CAT est prise en charge par une fonction spécifique facilitant l’agrégation des coûts de substitution lors du calcul de la distance d’édition globale, tandis que les méthodes EA, DAT et CombT doivent être entièrement gérées par l’utilisateur : la gestion de ces approches repose sur la manipulation d’autant d’instances d’objets séquences qu’il y a de domaines à analyser. Pour la méthode DAT, **TanaT** propose une approche plus intégrée grâce à une classe d’objets "trajectoires" qui agit comme un conteneur unifié regroupant plusieurs objets séquences (qui peuvent être de différents types d’entités et différents types de variables), permettant ainsi une automatisation du calcul des matrices de dissimilarité combinées. En revanche, les objets trajectoires de **TanaT** ne gèrent pas les stratégies EA, CAT et CombT, contraignant l’utilisateur à une implémentation manuelle impliquant, à l’instar de **TraMineR**, de jongler entre plusieurs objets séquences distincts. **sequenzo** se distingue sur ce point des deux autres bibliothèques en proposant une implémentation native des quatre approches mais sans la simplicité des objets trajectoires de **TanaT**.

2.3.6 Comparaison des performances de temps de calcul

L’une des critiques récurrentes adressées au langage R, et par extension à la bibliothèque **TraMineR**, concerne son coût computationnel [10]. L’optimisation des temps de calcul constitue ainsi l’un des enjeux majeurs de la transition de la communauté de l’analyse de séquences vers un écosystème Python, tel que souligné en introduction. Afin d’évaluer cet aspect, les performances temporelles des trois environnements ont été comparées pour la tâche de construction de typologies, dans des contextes unidimensionnels et multidimensionnels, à travers six tâches spécifiques. Les deux premières ont porté sur le calcul d’une distance d’Optimal Matching (OM) à coût constant et l’application d’un partitionnement hiérarchique (AGNES) selon la méthode de Ward sur les données **biofam** et les quatre suivantes sur la mise en œuvre des méthodes EA, CAT, DAT et CombT sur le jeu de données **extract_shp**, en combinant distance par Optimal Matching [11] et classification hiérarchique par lien complet. Les résultats sont consignés dans la Table 8 en annexe.

À l’exception du clustering avec les approches CAT et CombT pour **sequenzo** (respectivement 462 secondes

contre 1 pour **TanaT**, et 11 pour **TraMineR** et 13 secondes contre 5 pour **TanaT** et 4 pour **TraMineR**) et le clustering avec l’approche DAT pour **TanaT** (15 secondes contre environ 2 pour **TraMineR** et **sequenzo**), les trois bibliothèques présentent des résultats similaires. Il n’y a donc a priori pas de bibliothèque à favoriser dans tous les cas de figure.

2.3.7 Autres types d’analyses de séquences

Outre la construction de typologies, les environnements de **TraMineR** et **TanaT** proposent également d’autres outils d’analyse statistique des séquences, avec toutefois des niveaux de développement sensiblement différents. À ce titre, l’écosystème de **TraMineR** apparaît comme le plus complet. Il permet notamment d’étudier la variabilité des trajectoires à partir de leurs dissimilarités, en mesurant leur divergence globale²⁴, en évaluant la part de cette divergence associée à des groupes prédéfinis ou encore en construisant des arbres de divergence [12]. La comparaison de sous-groupes peut également s’appuyer sur un test du rapport de vraisemblance ou sur la différence de critère d’information bayésien (BIC) [13]. L’écosystème de **TraMineR** facilite par ailleurs l’articulation de l’analyse de séquences avec d’autres méthodes statistiques : les données peuvent notamment être converties dans des formats adaptés à l’analyse de survie ou préparées pour des approches portant sur les transitions et les sous-séquences, telles que la CTA, la SAMM ou la SHA²⁵, dont l’analyse repose ensuite sur des packages tiers [2].

Les possibilités offertes par **TanaT** sont plus limitées et se concentrent principalement sur l’analyse de survie. La bibliothèque permet notamment d’ajuster des modèles de type Coxnet et des arbres de survie²⁶, tout en pouvant être combinée à des bibliothèques spécialisées telles que **scikit-survival**.

Enfin, dans la version étudiée dans ce comparatif, **sequenzo** ne propose pas d’outils d’analyse statistique supplémentaires au-delà des fonctionnalités présentées précédemment.

2.3.8 Outils de traitement des autres types de séquences

Le dernier point de cette comparaison entre les environnements de **TraMineR**, **sequenzo** et **TanaT** porte sur la gestion de données autres que des séquences d’états : les séquences d’intervalles et d’événements définies en introduction. Ces deux types de séquences sont traités de manière très différente entre l’extension **TraMineRextras** et l’architecture native de **TanaT** et ne sont pour le moment pas pris en charge par **sequenzo**.

Dans l’environnement R, **TraMineRextras** introduit des outils spécifiques pour l’analyse de séquences d’événements uniquement, définies comme des successions ordonnées de transitions où chaque transition est composée d’une liste non ordonnée d’événements. Ce cadre permet le calcul de distances prenant explicitement en compte l’intervalle temporel entre les occurrences ainsi que la typologie des événements. Des outils statistiques permettent également la comparaison de ces séquences selon l’appartenance à des groupes prédéfinis [14].

Quant à lui, **TanaT** intègre nativement la manipulation des séquences d’intervalles et d’événements par le biais d’objets séquence dédiés (**EventSequencePool** et **IntervalSequencePool**). L’intérêt principal de cette architecture réside dans l’homogénéité des méthodes : la majorité des fonctions disponibles pour les séquences d’états sont applicables aux séquences d’intervalles et d’événements, et réciproquement. Cette cohérence simplifie considérablement la conduite des analyses et le travail simultané sur différents types de séquences, les appels de fonctions demeurant identiques. Toutefois, une limite analytique importante de l’implémentation de **TanaT** concerne la prise en compte de l’information temporelle dans le calcul des distances. Pour des séquences d’intervalles, une mesure de dissimilarité devrait idéalement pouvoir distinguer plusieurs types de modifications : un décalage temporel des intervalles, un changement de leur type ou encore une modification de leur durée, comme illustré

24. La divergence globale d’un ensemble de séquence mesure la variabilité entre les séquences. Par exemple, une divergence élevée, reflète une forte incertitude de la trajectoire suivie par les individus. Formellement, en notant w_i le poids de redressement de l’individu i , W la somme des poids de tous les individus et d_{ij} la dissimilarité entre les individus i et j , la divergence globale est définie comme $s^2 = \frac{1}{2W^2} \sum_{i=1}^n \sum_{j=1}^n w_i w_j d_{ij}$ [12].

25. **CTA** : Competing Trajectory Analysis ; **SAMM** : Sequence Analysis Multistate Model ; **SHA** : Sequence History Analysis

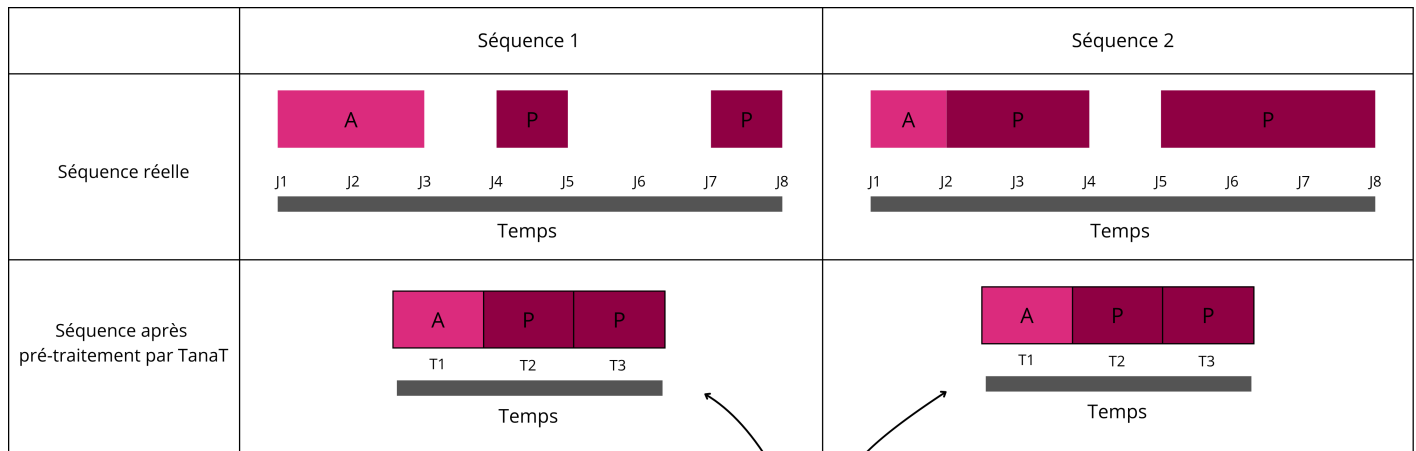
26. Les arbres de survie étendent les arbres de décision aux données de survie : à chaque nœud, une variable et un seuil sont choisis afin de séparer les individus en groupes présentant des profils de survie différents, tout en prenant en compte la censure.

par différentes variations d’une séquence de référence dans la Figure 14 en annexe. De même, pour des séquences d’événements dans la Figure 15 en annexe, il serait souhaitable de pouvoir distinguer un changement de type d’événement d’un simple décalage de sa date de survenue, à l’instar de certaines distances proposées dans **TraMineRextras**.

Afin d’évaluer si les distances disponibles dans **TanaT** sont sensibles à ces différentes transformations, les séquences de référence et leurs versions modifiées, telles que présentées dans les Figures 14 et 15, ont été comparées. Ces comparaisons ont été réalisées pour les sept mesures de distance actuellement implémentées dans la librairie: χ^2 , Dynamic Time Warping (DTW), Soft-Dynamic Time Warping (Soft-DTW), Linear Pairwise, Optimal Matching (OM), Longest Common Prefix (LCP) et Longest Common Suffix (LCS)²⁷. Les résultats sont présentés dans les Tables 9 et 10 en annexe.

Pour les séquences d’intervalles (Table 9 en annexe), les résultats sont particulièrement nets : toutes les distances détectent un changement de type d’intervalle, mais aucune n’est sensible à un simple décalage temporel. Ainsi, une séquence dont les intervalles ont uniquement été déplacés dans le temps présente une distance nulle avec la séquence de référence, comme si les deux séquences étaient identiques. Les modifications de durée sont également ignorées par l’ensemble des mesures, à l’exception de la distance du χ^2 . Cette dernière reste toutefois limitée pour la comparaison de séquences : elle repose uniquement sur les proportions associées à chaque type d’intervalle et ne tient pas compte de leur ordre d’occurrence, pourtant potentiellement déterminant dans l’analyse des trajectoires. Le même phénomène apparaît pour les séquences d’événements (Table 10 en annexe). Un changement de type d’événement est bien détecté, mais un décalage temporel seul ne produit aucune différence mesurable. De plus, lorsque le changement de type est combiné à un décalage temporel, les distances obtenues sont identiques à celles observées pour le seul changement de type. La position temporelle des événements n’intervient donc pas dans le calcul de ces mesures.

L’examen du code source de **TanaT** permet d’expliquer ce résultat. Pour l’ensemble des distances à l’exception du χ^2 , les séquences d’intervalles et d’événements font l’objet d’un pré-traitement qui les transforme en séquences d’états avant le calcul de la distance. Cette transformation conserve l’ordre et le type des éléments, mais supprime l’information portant sur leur position temporelle et, dans le cas des intervalles, sur leur durée comme le schématise la Figure 4. Ainsi, bien que **TanaT** permette de représenter des séquences d’intervalles et d’évènements, il n’existe aucune distance implémentée capable prendre en compte à la fois le décalage temporel et la durée pour les intervalles et le changement de type, comme l’utilisateur pourrait le souhaiter.



Pour TanaT, les séquences 1 et 2 sont identiques même si les séquences réelles sont totalement différentes. Les séquences sont déformées vers des séquences d’états en perdant toute information sur le temps entre les éléments ou leur durée.

FIGURE 4 – Schéma de l’effet du pré-traitement de TanaT sur les séquences d’intervalles vers des séquences d’états déformées

27. Pour les mesures LCP et LCS, une valeur élevée indique au contraire une plus forte similarité. Dans les Tables 10 et 9, une valeur de 1 correspond ainsi à des séquences considérées comme identiques.

2.4 Discussion

L'examen des possibilités offertes respectivement par **TraMineR**, **TanaT** et **sequenzo** dans la première partie de ce stage a permis de mettre en évidence des différences notables entre les trois écosystèmes.

TraMineR se positionne comme l'environnement le plus complet des trois. Il propose l'éventail le plus large de fonctionnalités, allant des statistiques descriptives aux analyses avancées, en passant par de multiples calculs de distances. Son architecture s'avère robuste et suffisante pour gérer des approches multidimensionnelles bien qu'un peu lourde. Sa principale limite est l'absence de prise en charge native des séquences d'intervalles, ce qui le limite conceptuellement par rapport à **TanaT**.

Toutefois, la proposition de **TanaT** est en demi-teinte. La librairie propose une infrastructure flexible pensée pour gérer simultanément les séquences d'états, d'évènements et d'intervalles, offrant une grande liberté à l'utilisateur au travers de l'implémentation de mesures de distance personnalisées, y compris des distances entre entités adaptées à des variables quantitatives. Cependant, ces atouts restent inexploités dans l'état actuel de la librairie : les distances implémentées par défaut ne permettant pas de différencier correctement des séquences que l'on souhaiterait pouvoir distinguer, et ne conviennent finalement réellement qu'aux séquences d'état. L'utilisateur doit nécessairement implémenter ses propres distances pour traiter des séquences d'intervalles ou d'évènements. Par ailleurs, la notion d'objet trajectoire regroupant plusieurs séquences par individus, pourtant intéressante, reste peu exploitée en dehors de la prise en charge du *Distance Additive Trick* (DAT) pour les analyses multidimensionnelles. En pratique, l'utilisateur se retrouve à gérer plusieurs objets séquences distincts pour implémenter les autres approches *multichannel*, tout comme avec **TraMineR**.

Enfin, **sequenzo** se distingue par la richesse de son catalogue de distances, incluant des mesures absentes de l'écosystème **TraMineR**, et par l'implémentation native des quatre approches de traitement multidimensionnel. En contrepartie, la librairie présente des lacunes et défauts importants : absence de gestion native des données, pas d'autres fonctionnalités que la construction de typologies, mais surtout des performances temporelles nettement inférieures pour l'approche *Cost Additive Trick* (CAT) en comparaison à **TraMineR** et **TanaT**.

Pour l'intégration du modèle **SalTo**, reposant sur le traitement conjoint de séquences d'évènements et d'états, l'utilisation de **sequenzo** est d'emblée écartée par l'absence de prise en charge d'autres types de séquences que les séquences d'états. Le constat est similaire pour **TanaT**, qui ne propose pas encore de métriques de distance adaptées aux séquences d'évènements. En revanche, **TraMineR** offre une solution technique viable en permettant de représenter et d'analyser ces deux types de séquences. Les possibilités restent toutefois limitées : il est actuellement possible d'analyser séparément les séquences d'états et d'évènements, ou de les combiner dans le cadre d'une analyse *multichannel* à l'aide d'approches telles que DAT ou CombT, mais aucun outil ne permet d'aller plus loin dans la modélisation explicite des liens entre les évènements et les états. Par ailleurs, la littérature ne propose pas encore d'application *multichannel* combinant directement des séquences d'états et des séquences d'évènements, ce qui limite également le recul disponible sur le fonctionnement et l'interprétation de telles analyses. Si l'analyse statistique devait dépasser la combinaison ou l'étude séparée de ces deux composantes, les fonctionnalités actuelles de **TraMineR** pourraient donc s'avérer insuffisantes.

Au terme de cette comparaison, **TraMineR** s'impose ainsi comme l'environnement à privilégier. Le recours à **sequenzo** ne semble justifié que si l'analyse exige une métrique absente de **TraMineR**. **TanaT** peut se montrer intéressant pour un utilisateur souhaitant implémenter ses propres distances, travailler avec des données quantitatives ou travailler uniquement avec Python. Cependant, ses pistes prometteuses pour les séquences d'évènements et d'intervalles ne sont malheureusement pas exploitables en l'état et ne compensent pas la perte des outils descriptifs et de visualisation présents dans **TraMineR**.

Dans cette perspective, il paraît plus pertinent d'encourager la mise à jour de **TraMineR** lui-même, en y intégrant les distances manquantes couvertes par **sequenzo** et **TanaT**, et en proposant une architecture supportant les séquences d'intervalles avec des distances adaptées. Cette voie semble d'autant plus pertinente que **TraMineR** reste l'outil de référence de la communauté de l'analyse de séquences : ses conventions, ses formats de données et ses fonctions sont largement intégrés dans les pratiques existantes, et migrer vers l'une des deux librairies alternatives impliquerait une rupture avec cet écosystème établi, sans gain fonctionnel ou temporel marqué en retour. Il convient néanmoins de rappeler que **sequenzo** et **TanaT** sont des bibliothèques (très) récentes par rapport à **TraMineR** qui sont encore en cours de développement actif, et que leurs propres trajectoires respectives

peuvent amener à revoir cette comparaison.

3 Construction de typologies de séquences : comparaison entre méthodes classiques et deep clustering

3.1 Problématique

La construction de typologies est une des tâches principales en analyse de séquences. Comme présenté précédemment lors de la comparaison des environnements disponibles, elle consiste à regrouper des séquences présentant des similarités afin d’identifier des profils types, dont l’appartenance peut ensuite être mise en relation avec différentes covariables.

Ces approches soulèvent néanmoins plusieurs difficultés. Tout d’abord, la typologie obtenue dépend d’une succession de choix méthodologiques qui peuvent chacun influencer les résultats : la mesure de dissimilarité utilisée, la stratégie retenue pour combiner les différentes variables (EA, CAT, DAT ou CombT), puis l’algorithme de clustering appliqué. À cette sensibilité méthodologique s’ajoute un manque de recul sur les situations où le nombre de variables est important. En effet, jusqu’à présent, les applications multidimensionnelles présentées dans la littérature reposent sur un nombre relativement limité de variables, ce qui implique que le comportement de ces approches lorsque le nombre de variables devient important reste encore peu documenté. Enfin, l’augmentation du nombre d’individus peut soulever des contraintes computationnelles importantes pour les méthodes fondées sur le calcul de matrices de dissimilarités, même à partir de quelques milliers d’individus.

À la vue de ces limitations, des méthodes issues de l’intelligence artificielle, et plus particulièrement le deep clustering, pourraient constituer une approche complémentaire. Contrairement aux méthodes précédentes, qui reposent sur la définition explicite d’une mesure de dissimilarité entre les trajectoires, les approches de deep clustering cherchent à apprendre dans un premier temps une représentation des données adaptée à leur classification, en se basant sur une architecture d’auto-encodeur. Il s’agit d’un réseau de neurones entraîné à reconstruire ses données d’entrée. Il se compose d’un encodeur, qui transforme les données en une représentation latente de plus faible dimension, et d’un décodeur, qui cherche à reconstruire les données d’origine à partir de cette représentation. La minimisation de l’erreur de reconstruction conduit ainsi le réseau à apprendre une représentation latente synthétisant une partie de l’information contenue dans les données initiales. Un algorithme de clustering est alors appliqué sur cet espace latent [15]. Grâce aux transformations apprises par les réseaux de neurones, ces modèles pourraient notamment permettre de représenter des structures plus complexes entre les différentes dimensions des trajectoires et amener à de meilleures performances pour la construction de typologies.

La seconde problématique de ce stage a donc consisté à déterminer dans quelle mesure des approches de deep clustering peuvent constituer une alternative ou un complément aux méthodes classiques de construction de typologies pour l’analyse de séquences multidimensionnelles. L’hypothèse étudiée est que l’apprentissage conjoint d’une représentation latente des trajectoires et de leur structure de classification peut, dans certaines configurations, conduire à des typologies plus proches de la structure sous-jacente des données que les approches classiques fondées sur des matrices de dissimilarité. Afin d’évaluer cette hypothèse, un protocole de comparaison a été mis en place entre plusieurs approches classiques et des méthodes de deep clustering sélectionnées dans la littérature. Leur comportement a d’abord été étudié sur des données simulées, permettant de contrôler la structure des trajectoires et différents niveaux de perturbation, puis les méthodes ont ensuite été appliquées à des données réelles afin d’évaluer leur comportement dans un contexte d’analyse pratique.

3.2 Méthodes et données

3.2.1 Méthodes comparées

Pour les approches classiques, la comparaison a été volontairement restreinte à une seule mesure de dissimilarité, une seule stratégie de combinaison multidimensionnelle et deux algorithmes de clustering. Il aurait bien sûr été possible d’explorer un plus grand nombre de distances, de stratégies *multichannel* et d’algorithmes de classification, mais l’objectif ici n’était pas de réaliser un benchmark exhaustif des approches classiques mais

plutôt d'évaluer si les approches de deep clustering apporteraient un gain par rapport à des choix d'analyse de séquences classiques.

Pour le calcul des dissimilarités, l'Optimal Matching (OM) avec des coûts de substitution fondés sur les taux de transition (TRATE)²⁸ a été retenu, notamment parce qu'il constitue l'une des approches les plus couramment utilisées en analyse de séquences [1].

Concernant la combinaison des différentes dimensions, aucune des quatre stratégies considérées (EA, CAT, DAT ou CombT) ne semble pouvoir être privilégiée de manière générale : la littérature souligne qu'il n'existe pas de stratégie systématiquement supérieure et que les typologies obtenues peuvent dépendre sensiblement de la manière dont les variables sont combinées dans le cadre du *multichannel* [9]. Dans ce contexte, l'approche DAT a été retenue comme une approche classique et établie de l'analyse de séquences multidimensionnelles, sans supposer qu'elle amènerait nécessairement à de meilleures performances que les autres approches.

Enfin, deux algorithmes de clustering couramment utilisés ont été choisis : *Partitioning Around Medoids* (PAM) et un algorithme de classification ascendante hiérarchique (CAH) avec le critère de Ward²⁹. Afin de diminuer le temps de calcul, le clustering hiérarchique n'a été appliqué qu'après une première étape de pré-classification avec l'algorithme PAM, permettant d'obtenir 100 points représentatifs. L'approche reposant sur PAM sera ci-après appelée DAT-PAM et celle reposant sur le clustering hiérarchique DAT-WARD.

Les approches de deep clustering ont été sélectionnées à partir d'une revue générale des méthodes existantes dans ce domaine [15]. La sélection a privilégié les méthodes capables de prendre en compte soit la dimension temporelle des données, soit leur caractère multidimensionnel. Ces critères ont fortement réduit le nombre de méthodes candidates : aucune des approches identifiées n'était directement conçue pour des séquences d'états qualitatifs telles que celles mobilisées en analyse de séquences. Deux méthodes initialement développées pour des données quantitatives ont donc été retenues puis adaptées : *Deep Multivariate Time Series Embedding Clustering* (DeTSEC) [16], qui exploite explicitement la structure temporelle des données, et *Single Cell Multimodal deep clustering* (scMDC) [17], davantage centré sur l'intégration de plusieurs variables.

L'approche DeTSEC repose sur un auto-encodeur intégrant des unités GRU (*Gated Recurrent Unit*³⁰). Son encodeur comprend deux réseaux GRU parcourant chaque série respectivement dans le sens chronologique et dans le sens inverse. Un mécanisme d'attention pondère ensuite les états cachés associés aux différents instants, avant que les représentations issues des deux directions ne soient combinées par un mécanisme de *gating* pour former la représentation latente de la trajectoire. Le décodeur suit une architecture symétrique et cherche à reconstruire la série dans les deux directions. L'apprentissage est réalisé en deux phases : l'auto-encodeur est d'abord pré-entraîné à partir des deux objectifs de reconstruction (sens chronologique et inverse), puis une fonction de coût liée au clustering est ajoutée. Un k-means appliqué aux représentations latentes fournit alors les affectations et les centroïdes, et le réseau est optimisé afin de rapprocher les observations du centroïde de leur groupe tout en maintenant les objectifs de reconstruction. La représentation latente est ainsi progressivement adaptée à la tâche de clustering.

À l'inverse, scMDC repose sur une architecture plus classique avec un auto-encodeur composé de couches denses mais conçu pour l'intégration de données multimodales. Les différentes variables sont concaténées et transmises à un encodeur commun afin de construire une représentation latente commune, tandis que des décodeurs distincts reconstruisent chacune des variables. Cette organisation vise à préserver l'information spécifique aux différentes variables tout en apprenant une représentation commune. Le pré-entraînement repose d'abord sur les pertes de reconstruction de chaque variable, auxquelles est ensuite ajoutée une perte fondée sur la divergence de Kullback-

28. Le coût de transition TRATE définit les coûts de substitution entre états à partir des fréquences de transition observées dans les données : deux états entre lesquels les transitions sont fréquentes sont considérés comme plus proches et reçoivent donc un coût de substitution plus faible, tandis que les transitions rares conduisent à des coûts plus élevés.

29. Contrairement à PAM, pour lequel le nombre de groupes K doit être fixé avant l'exécution de l'algorithme, la CAH construit une hiérarchie complète des observations sous la forme d'un dendrogramme, qui peut ensuite être coupé a posteriori au niveau correspondant au nombre de groupes retenu.

30. Une *Gated Recurrent Unit* (GRU) est une unité de réseau de neurones récurrent conçue pour traiter des données séquentielles. Elle utilise des mécanismes de *gate* (« porte ») pour déterminer, à chaque étape, quelles informations issues des étapes précédentes doivent être conservées, mises à jour ou oubliées, ce qui lui permet de modéliser des dépendances temporelles sur des séquences relativement longues.

Leibler afin de structurer progressivement l’espace latent pour la classification. Un k-means est alors utilisé pour initialiser les centroïdes des groupes avant une seconde phase d’apprentissage conjoint, combinant reconstruction, divergence de Kullback-Leibler et objectif de clustering.

Les architectures et procédures d’apprentissage ont suivi celles proposées dans les articles originaux [16, 17], à l’exception des adaptations nécessaires à la nature qualitative des données étudiées. Les états catégoriels ont ainsi été transformés par *one-hot encoding*³¹ et les fonctions de reconstruction originales ont été remplacées par une entropie croisée binaire calculée à partir des logits (*binary cross-entropy with logits*)³². Par ailleurs, les données considérées ici présentant des caractéristiques très différentes de celles utilisées dans les articles d’origine, notamment en raison du nombre de variables, du nombre de points temporels et de l’augmentation de la dimension des entrées induite par le *one-hot encoding*, aucun choix d’hyperparamètres ne pouvait être directement déduit des différentes configurations proposées par les auteurs. Les configurations associées aux jeux de données de plus grande dimension dans les articles originaux ont donc été retenues. Pour DeTSEC, la dimension de l’espace caché a été fixée à 512, la taille des *batches* à 16 et le taux d’apprentissage à 10^{-4} , avec 50 époques de pré-entraînement et 250 époques de raffinement. Pour scMDC, la dimension de l’espace caché a été fixée à 64, la taille des *batches* à 256 et le taux d’apprentissage du pré-entraînement a été fixé à 10^{-3} , avec 400 époques de pré-entraînement et 500 époques de raffinement ; les autres hyperparamètres ont été fixés à $\gamma = 0,1$, $\phi = 0,005$, $\tau = 0,1$ et $\alpha = 2$, avec un seuil de convergence de 0,001.

3.2.2 Données simulées

La comparaison des quatre approches nécessitait de disposer de données de séquences multidimensionnelles pour lesquelles la partition de référence était connue. À notre connaissance, aucun jeu de données labellisé présentant ces caractéristiques n’était disponible dans la littérature. Le recours à des données simulées a donc été privilégié afin de contrôler à la fois la structure des groupes et le niveau de perturbation appliqué aux trajectoires. La procédure de génération s’est appuyée sur la méthodologie proposée par Barban et Billari [18], initialement développée dans un cadre unidimensionnel et adaptée ici au cas multidimensionnel car la littérature ne propose pour l’instant pas d’approche de simulation pour le cas multidimensionnel à notre connaissance.

Dans sa formulation originale, cette méthodologie consiste à définir une séquence de référence pour chaque groupe, puis à la répliquer pour l’ensemble des individus appartenant à ce groupe. La même perturbation est ensuite appliquée indépendamment à chaque séquence selon un type et un niveau p de perturbation. Parmi les cinq types de perturbation proposés par les auteurs [18], trois ont été retenus car ils permettent de reproduire des situations susceptibles d’être rencontrées dans en analyse de séquences :

1. **Repoussement.** Avec une probabilité p , le statut observé au temps t est copié au temps $t + 1$. Les individus peuvent repousser de quelques années certains événements sans que leur trajectoire soit fondamentalement différente. Un exemple d’application de cette perturbation est donné dans la Figure 16.
2. **Mutation.** Avec une probabilité p , le statut observé au temps t est remplacé par un état tiré aléatoirement dans l’alphabet associé à la variable. Cette perturbation permet de représenter des erreurs de mesure. Un exemple d’application de cette perturbation est donné dans la Figure 17.
3. **Troncation.** Avec une probabilité p , la trajectoire est interrompue à partir d’un instant $t > 15$ tiré aléatoirement. Ce mécanisme reproduit des situations fréquentes dans les données longitudinales, dans lesquelles la durée d’observation varie entre individus ou certaines trajectoires sont censurées à droite. Un exemple d’application de cette perturbation est donné dans la Figure 18.

Afin de se rapprocher de données multidimensionnelles de trajectoires telles que celles considérées dans le modèle SaLTto ou dans le Swiss Household Panel, quatre profils de référence ont été construits manuellement. Ils reposaient sur trois grandes thématiques (familiale, résidentielle et qualité de vie) déclinées en dix variables : nombre d’enfants, statut marital, région d’habitation, type de commune, type de logement, qualité du logement, statut

31. Le *one-hot encoding* consiste à représenter une variable catégorielle par un vecteur binaire comportant une composante par modalité, seule celle correspondant à la modalité observée prenant la valeur 1.

32. La *binary cross-entropy with logits* combine une fonction sigmoïde et une entropie croisée binaire appliquées directement aux logits. Pour une cible binaire $y \in \{0, 1\}$ et un logit z , elle s’écrit $\mathcal{L}(y, z) = -[y \log(\sigma(z)) + (1 - y) \log(1 - \sigma(z))]$, avec $\sigma(z) = 1/(1 + e^{-z})$.

d’occupation du logement, satisfaction financière, départ en vacances et nombre mensuel de sorties entre amis. Les quatre profils type ont été définis de manière à représenter des trajectoires de vie plausibles et suffisamment différenciées. Ils sont présentés dans la Figure 19 en annexe.

Chaque groupe comprenait 2,500 individus, soit un effectif total de $N = 10,000$ trajectoires. Pour chaque individu et pour chacune des dix variables, un même type de perturbation a été appliqué au même niveau parmi $p \in \{0.1, 0.3, 0.6, 0.7, 0.8, 0.9, 0.99\}$. Cette sélection permettait de conserver des niveaux faibles, intermédiaires et d’observer plus en détail les changements de performances pour les niveaux de perturbation élevés. Le processus de génération a initialement été répété 100 fois pour chaque combinaison entre type et niveau de perturbation, afin de produire plusieurs réalisations indépendantes et d’évaluer la variabilité des résultats, mais les performances n’ont finalement été évaluées que sur les 5 premières à cause des contraintes temporelles du stage.

3.2.3 Métriques d’évaluation

L’évaluation sur données simulées a été menée en deux étapes. La première visait à caractériser l’effet des perturbations elles-mêmes sur les trajectoires et sur la structure des groupes simulés. Le nombre de transitions, la longueur des séquences, l’entropie *within*, la volatilité³³ et la turbulence³⁴ ont ainsi été calculés [1]. L’*Average Silhouette Width* (ASW)³⁵ de la partition d’origine a également été utilisée pour suivre l’évolution de la structure des quatre groupes au fil de l’augmentation du niveau de perturbation.

La seconde étape a consisté à évaluer la capacité des quatre méthodes de clustering à retrouver ces groupes de référence. Quatre critères de concordance complémentaires ont été utilisés : l’*accuracy*³⁶, l’*Adjusted Rand Index* (ARI), la *Normalized Mutual Information* (NMI) et l’*Adjusted Mutual Information* (AMI). L’*accuracy* mesure directement la proportion d’individus affectés au groupe de référence correspondant, tandis que l’ARI évalue la concordance des partitions à partir des paires d’individus placées ou non dans un même groupe. La NMI adopte une approche différente, fondée sur la quantité d’information partagée entre les deux partitions, et l’AMI en constitue une version corrigée pour tenir compte de la concordance attendue par le hasard. L’utilisation conjointe de ces indicateurs a ainsi permis d’évaluer la similarité des partitions selon plusieurs perspectives et de limiter la dépendance des conclusions aux propriétés d’un critère particulier. L’*accuracy* et la NMI prennent des valeurs comprises entre 0 et 1, tandis que l’ARI et l’AMI peuvent également prendre des valeurs négatives lorsque la concordance est inférieure à celle attendue par hasard. Pour les quatre indicateurs, une valeur de 1 traduit une correspondance parfaite avec la partition simulée [21].

3.2.4 Application au Swiss Household Panel

Enfin, la comparaison a été étendue au jeu de données réel `extract_shp`, présenté dans la partie précédente, pour des nombres de clusters compris entre 2 et 20. Contrairement aux données simulées, aucune partition de référence n’était ici disponible. L’objectif n’était donc pas d’identifier une méthode produisant la « bonne » typologie, ni de conduire une analyse substantielle des groupes obtenus, mais d’évaluer dans quelle mesure les différentes méthodes conduisaient à des conclusions concordantes. Pour chaque méthode, plusieurs critères internes ont ainsi été examinés pour l’ensemble des partitions comprenant de $k = 2$ à 20 clusters. L’ASW évalue conjointement la cohésion des observations au sein de leur cluster et leur séparation vis-à-vis des autres clusters, une valeur élevée indiquant une partition mieux structurée. L’indice de Calinski-Harabasz repose sur le rapport entre la dispersion inter-clusters et la dispersion intra-clusters et favorise ainsi les partitions constituées

33. La volatilité mesure l’instabilité d’une séquence en combinant la fréquence des changements d’état et la diversité des états visités. Elle est d’autant plus élevée que les transitions et les états visités différents sont nombreux. Son calcul est détaillé dans [19].

34. La turbulence mesure la complexité d’une séquence en combinant la diversité de ses sous-séquences et la variabilité des durées passées dans les différents états. Une turbulence élevée traduit ainsi des parcours à la fois changeants et irréguliers. Son calcul est détaillé dans [19].

35. L’*Average Silhouette Width* est un indicateur de la qualité d’un partitionnement. Pour chaque individu, cet indicateur regarde si l’individu est proche des individus issus du même clusters (cohésion) et si l’individu est éloigné des individus des autres clusters (séparation). L’ASW est simplement la moyenne pour l’ensemble des points du cluster.[20]

36. L’*accuracy* est calculée après un appariement glouton des labels de clusters.

de groupes à la fois compacts et bien séparés ; il est également à maximiser [22]. Enfin, le C de Hubert compare quant à lui les dissimilarités observées entre les individus appartenant à un même cluster aux valeurs extrêmes qu’elles pourraient prendre pour un même nombre de paires intra-clusters : une valeur faible traduit donc une plus forte homogénéité interne des groupes [23]. Ces critères reposant sur des principes différents et pouvant privilégier des nombres de groupes distincts, leur examen conjoint permet de ne pas faire dépendre le choix du nombre de clusters optimal k d’un indicateur unique.

3.3 Résultats

3.3.1 Impact des perturbations

Les effets des trois perturbations sur les caractéristiques des séquences sont présentés pour chacun des niveaux de perturbation et selon les six indicateurs définis précédemment dans la Figure 5. Les cinq indicateurs calculés à l’échelle de chaque séquence (individu $\times$ variable) évoluent globalement dans le sens attendu. La mutation augmente fortement le nombre de transitions, l’entropie *within*, la volatilité et la turbulence, sans modifier la longueur des séquences. À l’inverse, le repoussement réduit progressivement leur complexité, avec une diminution des autres indicateurs lorsque le niveau de perturbation augmente. La troncature agit principalement sur la longueur des trajectoires, les autres caractéristiques restant relativement stables.

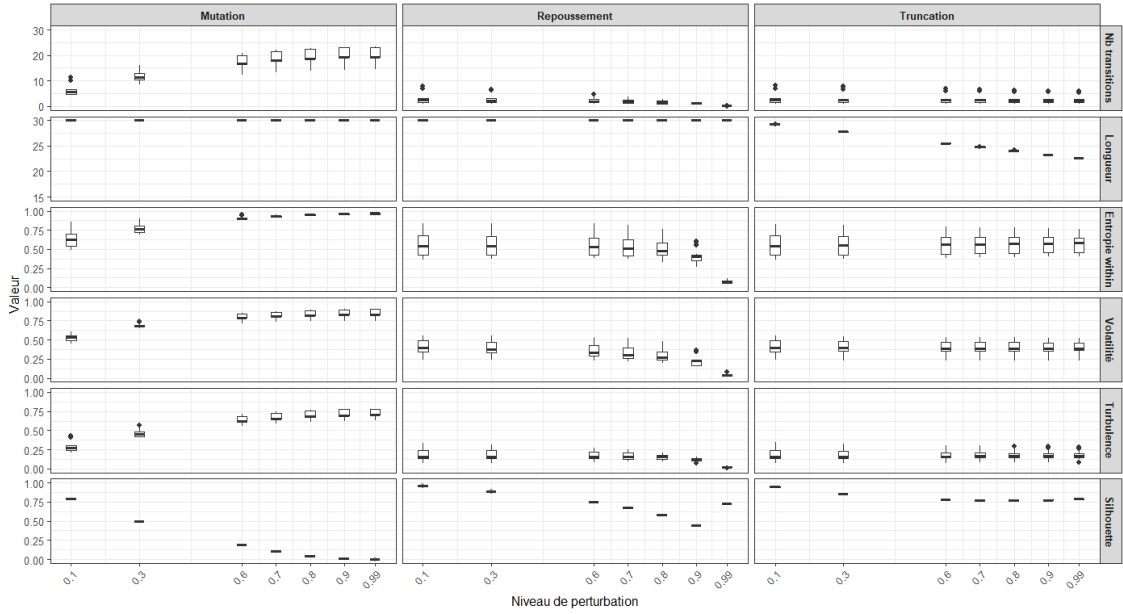


FIGURE 5 – Impacts des trois types de perturbation selon le niveau de perturbation sur le nombre de transitions, la longueur, l’entropie *within*, la volatilité, la turbulence et l’ASW des séquences pour les seeds 1 à 100

L’ASW de la partition d’origine permet d’examiner plus directement les conséquences de ces transformations sur la structure des quatre groupes. Pour la mutation, leur séparation se dégrade fortement et de manière plutôt monotone : l’ASW passe d’environ 0.8 à $p = 0.1$ à 0.5 dès $p = 0.3$, puis devient presque nulle aux niveaux les plus élevés. Les trajectoires s’éloignent ainsi progressivement de leur profil d’origine jusqu’à devenir essentiellement aléatoires. Le repoussement entraîne une dégradation plus progressive, la séparation restant perceptible jusqu’à $p = 0.9$. La remontée observée à $p = 0.99$ peut s’expliquer par le caractère presque systématique de la perturbation : les séquences tendent alors à répéter leurs premiers états, ce qui peut recréer une différenciation lorsque les profils se distinguent dès le début de la trajectoire. Enfin, la troncature n’entraîne qu’une diminution modérée de l’ASW, la partie initiale des séquences, qui porte déjà une part importante de la structure des groupes, étant conservée. Ainsi, contrairement à la mutation, le repoussement et surtout la troncature préservent une séparation relativement nette des groupes, y compris pour des niveaux élevés de perturbation.

3.3.2 Qualité du clustering

Ces différences de structure se retrouvent directement dans la capacité des quatre méthodes à récupérer la partition simulée, présentée dans la Figure 6. Pour le repoussement et la troncature, toutes obtiennent des performances quasi parfaites sur l'ensemble des niveaux étudiés, avec des valeurs d'ARI, de NMI, d'AMI et d'*accuracy* très proches de 1. Aucune méthode ne se distingue donc réellement dans ces deux scénarios.

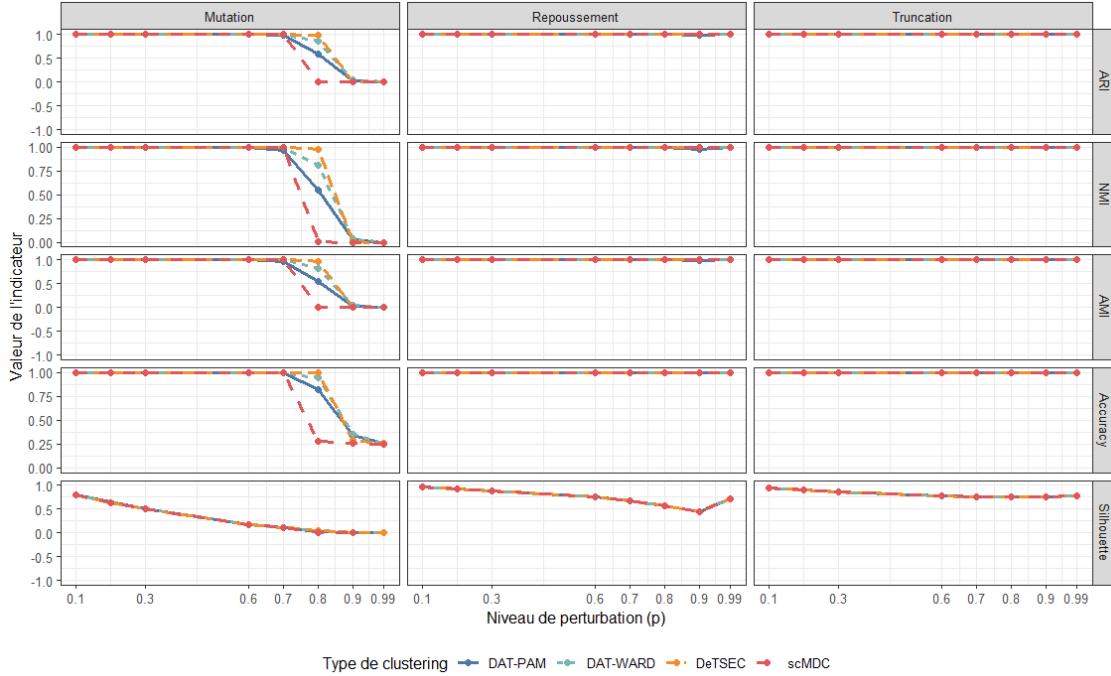


FIGURE 6 – Évolution de la qualité du partitionnement obtenu pour les approches classiques et de deep clustering selon le type et le niveau de perturbation

La mutation est la seule perturbation pour laquelle des différences apparaissent. Jusqu'à $p = 0.7$, les quatre méthodes retrouvent presque parfaitement les groupes simulés. À $p = 0.8$, les performances commencent à se différencier, avec une hiérarchie commune aux quatre indicateurs : DeTSEC obtient les meilleurs résultats, suivi de DAT-WARD, DAT-PAM puis scMDC. À partir de $p = 0.9$, l'ARI, la NMI et l'AMI deviennent proches de zéro, tandis que l'*accuracy* avoisine 0.25, soit le niveau attendu pour une attribution aléatoire entre quatre groupes. Cette chute intervient précisément lorsque l'ASW de la partition d'origine devient elle-même presque nulle. Les faibles performances observées à ces niveaux traduisent donc moins l'échec d'une méthode particulière que la quasi-disparition de la structure de groupes initialement simulée.

3.3.3 Application au Swiss Household Panel

En l'absence de partition de référence, l'analyse du jeu `extract_shp` s'est d'abord appuyée sur des critères internes de qualité pour orienter le choix du nombre de clusters. Les évolutions de l'ASW, du C de Hubert et de l'indice de Calinski-Harabasz sont présentées dans la Figure 7. Deux constats principaux se dégagent. D'une part, les valeurs de silhouette restent inférieures à 0,25 pour les quatre approches, quel que soit le nombre de groupes considéré, indiquant une structure de partitionnement peu marquée dans tous les cas. D'autre part, à l'exception de DeTSEC, les méthodes DAT-WARD, DAT-PAM et scMDC présentent des évolutions relativement similaires pour les indices de Calinski-Harabasz et le C de Hubert ; les deux approches classiques montrent également des profils assez similaires pour l'ASW.

Les différents critères ne conduisent toutefois pas au même choix de k . L'indice de Calinski-Harabasz, à maximiser, privilégie globalement de faibles nombres de groupes, autour de $k = 2$ ou 3 pour toutes les méthodes. Pour l'ASW, les maxima diffèrent davantage selon les méthodes : ils se situent autour de $k = 2$ pour les deux

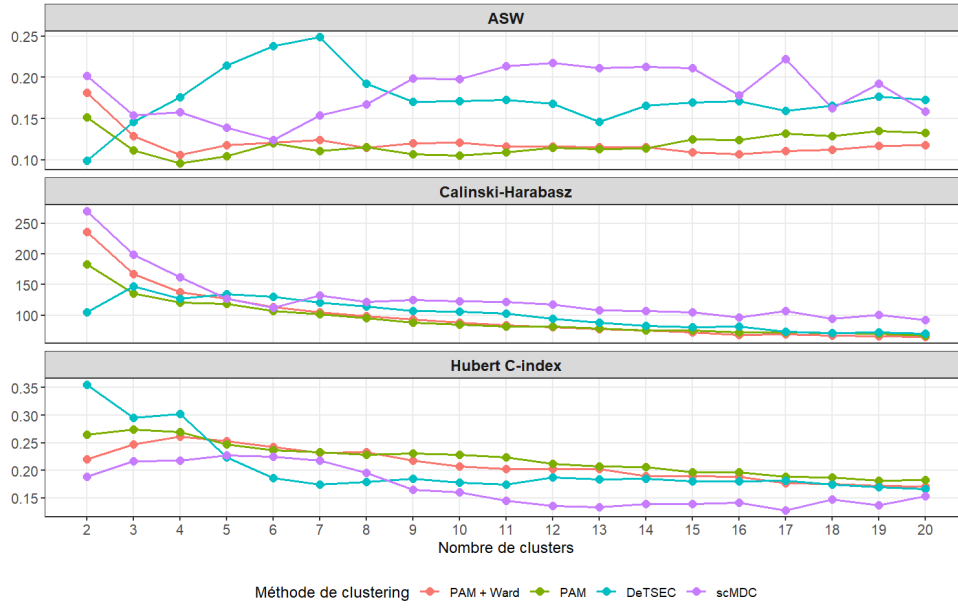


FIGURE 7 – Indicateurs de la qualité du partitionnement en fonction du nombre de cluster pour les quatre approches de clustering

approches DAT, de $k = 7$ pour DeTSEC et entre $k = 11$ et $k = 17$ pour scMDC. Le C de Hubert, à minimiser, conduit encore à d'autres choix : les valeurs les plus faibles sont obtenues pour des nombres de groupes proches de 20 avec les approches DAT, autour de $k = 7$ pour DeTSEC et de $k = 17$ pour scMDC. Aucun nombre de clusters ne s'impose donc clairement à partir des seuls critères internes. Dans un contexte réel sans partition de référence, le choix du nombre de cluster doit être complété par l'examen des groupes obtenus et mis en parallèle avec une expertise métier pour mesurer la pertinence et l'interprétabilité du partitionnement sélectionné.

3.4 Discussion

Sur les données simulées, les performances des quatre méthodes ont principalement dépendu du type et du niveau de perturbation appliqués. Pour le repoussement et la troncature, DAT-PAM, DAT-WARD, DeTSEC et scMDC ont retrouvé presque parfaitement les groupes simulés sur l'ensemble des niveaux étudiés, avec des valeurs d'ARI, de NMI, d'AMI et d'*accuracy* très proches de 1. Des différences substantielles ne sont apparues que pour la mutation à partir de $p = 0,8$, lorsque l'ASW de la partition simulée devenait elle-même presque nulle et que les groupes de référence étaient donc déjà très peu séparés. Il était donc difficile de faire une vraie distinction entre les performances des différentes approches à partir de ces résultats.

Ces derniers mettent avant-tout en évidence certaines limites du protocole de simulation retenu. Si l'approche proposée par [18] permettait de contrôler précisément la structure des groupes et les perturbations appliquées, le repoussement et la troncature se sont révélés trop peu discriminants pour départager les méthodes. À l'inverse, la mutation n'a fait apparaître des écarts marqués qu'à des niveaux de perturbation très élevés, lorsque la structure initiale était déjà fortement dégradée. Les faibles performances observées à partir de $p = 0,9$ traduisent ainsi davantage la quasi-disparition de cette structure que l'échec des méthodes. Dans les conditions étudiées, aucune supériorité nette des approches classiques ou de Deep Clustering n'a donc pu être mise en évidence, les écarts les plus importants apparaissant par ailleurs dans des situations dont la proximité avec des données longitudinales réelles peut être discutée.

L'analyse du jeu de données réel `extract_shp` a soulevé une difficulté différente. En l'absence de partition de référence, le choix du nombre de groupes reposait sur plusieurs critères internes, sans qu'une solution commune ne se dégage. Les valeurs d'ASW sont restées inférieures à 0,25 pour toutes les méthodes et tous les nombres de clusters considérés, témoignant d'une structure de groupes globalement peu marquée. Les différents indicateurs ont en outre conduit à des choix divergents.

Ces résultats ont finalement souligné une limite plus générale de la construction de typologies sur données réelles. Lorsque les trajectoires ne présentent pas naturellement de groupes bien séparés, un algorithme de clustering impose malgré tout une partition et plusieurs solutions concurrentes peuvent alors être défendables selon le critère retenu. Les indicateurs internes ne suffisent pas à identifier une partition optimale ; l'interprétation des groupes, leur stabilité et leur pertinence au regard du domaine d'application apparaissent alors comme des éléments essentiels. Dans les expériences menées ici, les approches de deep clustering n'ont pas permis de lever cette difficulté : elles n'ont pas produit des partitions différentes des approches classiques, sans faire apparaître une structure plus nette ou plus facilement interprétable.

4 Utilisation de l'Intelligence Artificielle

Au cours de ce stage, j'ai utilisé plusieurs outils d'IA générative, principalement les modèles GPT-5.4 et GPT-5.5 d'OpenAI et Gemini 3 Pro de Google. Cet usage a concerné en premier lieu la rédaction, pour reformuler des paragraphes préalablement rédigés, améliorer leur fluidité et corriger d'éventuelles fautes, chaque proposition étant ensuite systématiquement relue et adaptée. J'ai également utilisé l'IA comme assistance pour l'écriture du code pour des tâches génériques en R et Python, tandis que j'ai codé les parties spécifiques aux bibliothèques TraMineR, TanaT et sequenZo entièrement à partir de leur documentation. Pour les méthodes DeT-SEC et scMDC, je me suis servie de l'IA afin de produire une première structure de code PyTorch à partir des architectures que j'ai définies à la suite de mes lectures d'articles, avant une relecture et une correction détaillées. Enfin, j'ai utilisé l'IA pour écrire les scripts de soumission sur le cluster de calcul GRICAD. Les choix méthodologiques, l'analyse scientifique, l'interprétation des résultats et la structure du rapport restent de ma responsabilité, l'IA ayant essentiellement joué un rôle d'assistance à la formulation et au code.

5 Impact environnemental et social

5.1 Organisme d'accueil

Les deux organismes qui ont accueilli ce stage s'inscrivent dans un environnement institutionnel où les enjeux environnementaux et sociaux occupent une place croissante. Sur le plan environnemental, l'Université Grenoble Alpes (UGA) s'engage notamment dans une politique de transformation écologique visant à réduire son empreinte carbone, par exemple au travers d'un plan de mobilité favorisant les modes de transport bas carbone [24]. Cette orientation se retrouve également au sein des lieux où le stage s'est déroulé, le CReSI, qui accueille une partie du laboratoire TIMC, et l'IMAG, qui accueille une partie du LIG, ont par exemple été conçus selon des principes bioclimatiques et de performance énergétique [25]. Au-delà du fonctionnement des bâtiments, les enjeux environnementaux sont également intégrés aux activités de recherche et de sensibilisation, avec des travaux portant sur la compréhension des effets du changement climatique ou sur des systèmes numériques plus sobres, ainsi que des actions de sensibilisation à la sobriété numérique au sein des différents laboratoires grenoblois. Cette démarche reste néanmoins confrontée à une tension particulière, notamment dans le cas du LIG, la recherche en informatique contribue à rendre les systèmes numériques plus efficaces, mais participe parallèlement au développement continu de nouveaux usages et de capacités de calcul toujours plus importantes. L'amélioration de l'efficacité des méthodes ne conduit donc pas nécessairement à une diminution globale de leur impact, notamment lorsque les volumes de calcul augmentent simultanément. Cette tension entre progrès technologique et sobriété constitue ainsi un enjeu important pour la recherche en informatique [26, 27]. Sur le plan social, les deux laboratoires s'inscrivent également dans les politiques conduites par leurs tutelles. L'UGA dispose notamment d'un plan d'action pour l'égalité entre les femmes et les hommes pour la période 2025-2028, qui prévoit des actions concernant l'égalité des carrières et des rémunérations, la lutte contre les violences sexistes, sexuelles et les discriminations, la mixité des métiers ou encore l'articulation entre vie personnelle et professionnelle [28]. Ces démarches témoignent d'une volonté institutionnelle de réduire les inégalités, sans signifier pour autant que celles-ci aient disparues : comme dans l'ensemble du milieu académique et scientifique, les questions de représentation des femmes dans certaines disciplines, d'inégalités de carrière et, plus largement, de diversité sont toujours présentes dans ces structures [29, 30].

5.2 Enjeux environnementaux personnels

L'impact environnemental direct de ce stage reste relativement limité. Les trajets quotidiens vers mon lieu de stage ont été effectués à pied ou en transports en commun (bus ou tram). Le principal déplacement longue distance a concerné une conférence à Rennes, réalisée en train. À cela se sont toutefois ajoutés environ 240 km parcourus en voiture au cours de la semaine pour effectuer les trajets entre le lieu où j'étais hébergée et celui de la conférence. Enfin, les réunions ont eu lieu sur mes lieux de stage (CRESI ou IMAG), au BMD ou quelques fois à distance, ce qui a généré peu de déplacements supplémentaires, alors effectués en transports en commun. La majorité du travail a été réalisée sur un ordinateur portable, principalement pour du traitement de données, du développement de code et de la rédaction. Cet ordinateur n'ayant pas été acheté spécifiquement pour le stage, son coût environnemental de fabrication ne peut pas être directement attribué au projet. La consommation électrique liée à son utilisation est restée proche d'un usage informatique classique, même si certaines tâches particulièrement longues, notamment les simulations de données et certains clusterings sous R, ont nécessité de laisser l'ordinateur fonctionner toute la nuit.

La seconde partie du stage a également nécessité l'utilisation du cluster de calcul GRICAD, notamment pour l'entraînement et l'évaluation des méthodes de deep clustering. Cette utilisation constitue probablement l'un des principaux postes de consommation énergétique directement liés au projet, compte tenu des ressources de calcul mobilisées. Enfin, plusieurs services en ligne ont été utilisés au cours du stage, notamment des outils d'IA générative tels que ChatGPT et Gemini, comme présenté précédemment. Leur utilisation ponctuelle mobilise des ressources de calcul, auxquelles s'ajoute le coût environnemental non-négligeable lié à l'entraînement et au maintien en fonctionnement de modèles de grande taille même si celui-ci est dilué entre tous les utilisateurs. Les autres services utilisés sont notamment GitHub, pour l'hébergement des fichiers du projet, Notion, pour son suivi et Canva pour la réalisation de présentations et d'illustrations. L'ensemble de ces services repose sur des infrastructures de centres de données et contribue donc à l'empreinte environnementale globale du stage.

5.3 Enjeux environnementaux globaux

Si l'impact environnemental direct du projet reste modéré, il convient toutefois de considérer plus largement les pratiques méthodologiques auxquelles il se rattache, en particulier dans la seconde partie du stage consacrée au deep clustering.

L'analyse de trajectoires peut mobiliser des volumes de données importants, tandis que les approches fondées sur l'apprentissage profond tendent précisément à tirer parti de jeux de données toujours plus volumineux. Au-delà du coût propre à chaque étude, qui peut rester limité pris isolément, le développement, l'utilisation et la diffusion de ces méthodes participent ainsi à une généralisation de l'apprentissage profond dans des domaines de plus en plus nombreux. Cette dynamique contribue à faire de modèles fortement consommateurs de ressources une solution courante, parfois au détriment d'approches plus simples susceptibles d'offrir des performances comparables. Elle s'accompagne de besoins croissants en puissance de calcul, en stockage et en infrastructures de traitement, eux-mêmes associés à d'importantes consommations d'électricité et d'eau. Le coût environnemental d'une méthode ne peut donc être apprécié uniquement à l'échelle de son utilisation ponctuelle, mais doit également être replacé dans la dynamique plus générale que sa diffusion contribue à alimenter.

Les résultats obtenus au cours de ce stage invitent justement à nuancer l'intérêt d'un recours systématique aux méthodes d'apprentissage profond. Dans les expériences réalisées, les approches de deep clustering ne se montrent pas nécessairement plus performantes que les méthodes classiques pour des niveaux de perturbation pouvant être considérés comme réalistes. Dans le cas des perturbations par mutation, par exemple, des différences notables n'apparaissent qu'à partir de niveaux particulièrement élevés ($p \geq 0,8$).

Ces observations conduisent ainsi à mettre en regard le surcoût computationnel et environnemental des approches d'apprentissage profond avec les gains de performance qu'elles procurent effectivement. Un gain même faible peut naturellement être déterminant dans certaines situations à fort enjeu, notamment en santé, tandis qu'il sera plus difficile à justifier lorsque les conséquences d'une erreur sont limitées. Le choix d'une méthode ne devrait donc pas reposer uniquement sur sa sophistication ou sur ses performances maximales, mais également sur les ressources qu'elle mobilise et sur l'utilité réelle du gain obtenu. Dès lors, lorsque des méthodes plus simples permettent

d’atteindre des résultats comparables, le recours à des modèles plus coûteux devrait également être un objet de questionnement important du projet.

5.4 Enjeux sociaux globaux

À l’instar des enjeux environnementaux, les impacts sociaux directs de ce projet restent limités, ce stage n’ayant conduit au déploiement d’aucun outil d’analyse. Les méthodes étudiées s’inscrivent néanmoins dans le champ plus large de l’analyse de séquences, pouvant être mobilisée notamment pour étudier des trajectoires de vie, des parcours professionnels ou encore des trajectoires de santé. Les travaux présentés dans ce rapport ne soulèvent donc pas d’enjeu social immédiat, mais ils prennent place dans un ensemble de méthodes dont les applications peuvent, elles, avoir des conséquences importantes sur les individus et les populations.

Ces méthodes offrent tout d’abord des perspectives importantes pour mieux comprendre des phénomènes qui se construisent dans le temps. Dans le domaine de la santé, l’identification de profils de patients peut par exemple contribuer à mieux caractériser certains facteurs de risque, à mettre en évidence des associations entre des états éloignés dans le temps ou encore à améliorer l’anticipation et la prise en charge de certaines situations. De manière analogue, en sciences sociales, l’étude quantitative des trajectoires permet de faire apparaître des régularités dans les parcours scolaires, professionnels, familiaux ou résidentiels. Elle peut ainsi contribuer à documenter des mécanismes tels que la reproduction sociale ou, au contraire, à identifier des trajectoires s’écartant des parcours dominants et leurs origines. Dans cette perspective, la valeur de l’analyse réside avant tout dans la mise en évidence de tendances collectives et dans la compréhension de la diversité des parcours.

Cette capacité à faire émerger des structures communes repose toutefois nécessairement sur une simplification. Construire une typologie revient à résumer une pluralité de trajectoires individuelles complexes par un nombre limité de profils. Cette réduction est précisément ce qui rend les données compréhensibles, mais elle constitue également une première source de risque : deux individus affectés à un même groupe peuvent conserver des parcours, des contraintes et des perspectives différents. Les trajectoires atypiques peuvent également être moins bien représentées, même si certaines méthodes sont spécifiquement conçues pour les identifier, cet objectif restant secondaire dans une grande partie des usages de l’analyse de trajectoires [31]. Un profil statistique ne constitue donc ni une description exhaustive d’un individu, ni une prédiction certaine de son évolution future.

Le risque deviendrait plus important si cette simplification, pertinente à l’échelle d’une population, était transposée directement à l’échelle individuelle. Une interprétation trop déterministe des profils pourrait conduire à enfermer les individus dans des catégories statistiques et à transformer une association observée en présomption sur leur état futur ou leur niveau de risque. Dans des domaines tels que la santé ou pour l’accès à certains services, cette logique pourrait contribuer à des décisions défavorables voire discriminatoires si l’appartenance à un profil devenait un critère de sélection ou de restriction. Dans le domaine médical, par exemple, l’affectation d’un patient à un profil ne saurait se substituer à l’analyse de son dossier individuel, à l’expertise du professionnel de santé et au dialogue avec le patient. Ces méthodes doivent ainsi rester des outils d’aide à la compréhension ou à la décision, et non devenir des mécanismes automatisés assignant aux individus un devenir supposé.

À ces risques d’interprétation s’ajoutent ceux liés aux données nécessaires à de telles analyses. L’étude de trajectoires repose par définition sur l’accumulation d’informations concernant un même individu au cours du temps, informations qui peuvent être nombreuses et particulièrement sensibles lorsqu’elles concernent la santé, la situation familiale, professionnelle ou sociale. Leur utilisation soulève donc des questions relatives au consentement, à la protection de la vie privée, aux conditions d’accès et à la durée de conservation des données. Plus largement, la généralisation de ces approches peut contribuer à banaliser la collecte, le rapprochement et la centralisation de volumes croissants de données personnelles. Dans le contexte européen, leur traitement doit notamment respecter le cadre fixé par le règlement général sur la protection des données (RGPD). Une vigilance particulière doit également porter sur les infrastructures utilisées pour leur stockage et leur traitement, afin que les conditions de sécurité, d’accès et de gouvernance restent maîtrisées.

Les enjeux sociaux associés à ce travail résident donc moins dans ses conséquences immédiates que dans la manière dont les méthodes étudiées pourraient être mobilisées à plus grande échelle. L’analyse de trajectoires peut constituer un outil précieux pour mieux comprendre des phénomènes sociaux ou sanitaires et, dans certains cas, contribuer à leur prévention ou à une meilleure prise en charge. Ces bénéfices ne doivent toutefois pas occulter

les risques associés à leur généralisation et à certains usages, qui imposent de conserver un regard critique sur les conditions dans lesquelles ces méthodes sont utilisées, tant au niveau de la collecte et du traitement des données que de l'interprétation et de l'utilisation des résultats produits par les modèles.

6 Discussion et conclusion

Ce stage s'est articulé autour de deux niveaux d'étude de l'analyse de séquences. Le premier a consisté à dresser un état des possibilités actuellement offertes par les outils disponibles sous R et Python, au travers de la comparaison de **TraMineR**, **TanaT** et **sequenzo**. Celle-ci a mis en évidence que **TraMineR** s'impose actuellement comme l'environnement à privilégier pour l'implémentation du modèle **SalTo** et pour l'analyse de séquences, excepté dans quelques cas où les spécificités de **TanaT** et **sequenzo** seraient nécessaires à l'analyse. Le second volet de ce stage s'est concentré sur la problématique plus spécifique de la construction de typologies et sur l'intérêt des méthodes de deep clustering par rapport aux approches classiques dans un contexte multidimensionnel de grande dimension. Sur les données simulées, les quatre approches ont obtenu des performances très proches, les différences n'apparaissant que pour des niveaux de mutation très élevés. Sur le jeu de données **extract_shp**, aucune méthode n'a fait émerger une structure de partitionnement clairement supérieure ou plus stable que les autres. Dans l'ensemble, ces résultats ne mettent donc pas en évidence d'avantage net du deep clustering dans les configurations étudiées, sans qu'il soit possible de déterminer si cela tient principalement aux méthodes elles-mêmes ou plutôt aux jeux de données trop ou insuffisamment discriminants ou structurés. Ils suggèrent néanmoins que le recours à des architectures plus complexes n'est pas nécessairement justifié lorsque la structure des groupes est déjà correctement identifiable.

Ces deux volets ont présenté une limite commune, liée plus généralement à l'étude de trajectoires longitudinales : le manque de données. Trouver des séquences unidimensionnelles avec un nombre d'individus et de covariables satisfaisant s'est révélé relativement aisé, mais cela a été beaucoup plus fastidieux pour les séquences multidimensionnelles, dès lors qu'il fallait disposer de jeux de données riches, collectés sur une période suffisamment longue et comportant un nombre de pas de temps suffisamment important. Il s'agit en effet de données longues et coûteuses à construire et à maintenir, potentiellement sensibles, qui ne sont donc pas nécessairement facilement partagées, voire existantes. Dans le cas de l'évaluation des méthodes de construction de typologies, une difficulté supplémentaire a résidé dans l'absence totale, à ma connaissance, de jeux de données réels multidimensionnels pour lesquels une partition de référence serait disponible, ce qui a limité les possibilités d'évaluation externe et explique le recours à des données simulées pour cette partie.

Concernant plus spécifiquement la comparaison des bibliothèques, une autre limite concerne l'évaluation des performances, qui gagnerait à être reproduite sur d'autres jeux de données, de tailles et de domaines différents. Toutefois, la limite principale, apparue très concrètement au cours du stage, tient à la rapidité d'évolution des outils étudiés. Plusieurs mises à jour de **TanaT** et **sequenzo** ont eu lieu au cours des cinq mois de stage, modifiant substantiellement les fonctionnalités disponibles. Cette évolution rapide est évidemment encourageante pour le développement de l'analyse de séquences sous Python, mais elle rend également difficile l'établissement d'un état des lieux destiné à rester durablement à jour. Le travail réalisé devrait donc davantage être considéré comme un état des lieux des possibilités disponibles à un instant donné que comme une description définitive de ces différents écosystèmes. Une mise à jour de ce comparatif est ainsi envisagée dans les mois suivant le stage, avec notamment l'ajout de section concernant la gestion des données manquantes et des séquences de longueur différentes.

Le second volet laisse également plusieurs perspectives ouvertes. Sa réalisation dans la dernière partie du stage, combinée aux temps de calcul importants nécessaires aux simulations et à l'entraînement des modèles, a conduit à restreindre les analyses initialement envisagées. En particulier, les expériences n'ont finalement été conduites que sur 5 seeds, alors qu'il avait initialement été envisagé de tester les quatre approches sur 100 seeds afin d'évaluer plus précisément la variabilité des résultats. L'exploration des hyperparamètres a également été limitée : une étude plus complète serait nécessaire, avec notamment une sélection des meilleures configurations d'hyperparamètres, puis leur évaluation sur l'ensemble de la grille de perturbations. Pour **DeTSEC**, cette exploration

mériterait notamment d'être prolongée, certaines des meilleures configurations obtenues se situant aux bornes de la grille testée, ce qui suggère que des valeurs situées au-delà pourraient encore améliorer les performances. L'exploration des configurations est également restée limitée. Pour les approches de deep clustering, les résultats obtenus pourraient probablement être sensiblement modifiés par un réglage plus approfondi des hyperparamètres, qui n'a pu être mené de manière systématique dans le temps imparti. Une démarche analogue pourrait également être menée pour les approches classiques, non pas uniquement à travers leurs paramètres internes, mais en élargissant les choix méthodologiques considérés : autres mesures de dissimilarité, autres stratégies de combinaison multichannel ou autres algorithmes de classification. Les performances observées doivent ainsi être interprétées comme celles des configurations retenues dans cette étude, et non comme une comparaison exhaustive du potentiel maximal de chacune des familles de méthodes. De plus, il aurait été souhaitable d'intégrer une comparaison rigoureuse des temps de calcul mais celle-ci aurait nécessité l'exécution de l'ensemble des méthodes et des configurations sur un matériel informatique identique. Les calculs ayant été réalisés à la fois localement et sur différentes ressources du cluster de calcul GRICAD, cette comparaison n'a finalement pas pu être retenue. Cette situation illustre plus généralement un enjeu inhérent à l'analyse de séquences : lorsque les temps de calcul deviennent importants, les choix méthodologiques doivent également tenir compte des ressources disponibles et de la durée limitée du projet.

Avec le recul, les simulations constituent également un point important de cette analyse qu'il serait intéressant d'approfondir. Les perturbations de mutation, de troncature et de repoussement avaient été choisies afin de contrôler précisément les différents types et niveaux d'altération, tout en représentant des modifications réalistes des données. Cependant, l'examen des silhouettes des partitions a montré que, pour de nombreux scénarios, les groupes simulés restaient nettement séparés, en particulier pour le repoussement et la troncature ou à l'inverse qu'aucune structure ne subsistait pour les niveaux de perturbations les plus élevés de mutation. Les résultats obtenus restent pertinents pour une première évaluation, mais ils ne couvrent qu'une partie des configurations susceptibles d'être rencontrées en pratique. Une poursuite de ce travail pourrait donc s'appuyer sur une application simultanée de plusieurs types de perturbations ou de différents types de perturbations selon les individus pour induire plus de variété ou s'orienter vers d'autres approches de génération, comme des chaînes de Markov, afin d'étudier les méthodes dans des situations où la structure latente est moins immédiatement identifiable.

D'autres pistes méthodologiques sont également apparues au fil des échanges menés pendant le stage. Une alternative intéressante consisterait notamment à étudier des méthodes de réduction de dimension telles qu'UMAP³⁷, à partir non pas directement des séquences brutes, mais de matrices de dissimilarités obtenues avec des approches telles que EA, CAT ou DAT. Cela permettrait de tout à la fois préserver la structure temporelle et multidimensionnelle des données, tout en recherchant une représentation de dimension réduite susceptible de faciliter la construction ou l'interprétation des typologies sans recourir à des algorithmes aussi complexes que ceux du deep clustering. Enfin, la question de la grande dimension pourrait être étendue au-delà du seul nombre d'individus. Dans un contexte où les dispositifs de collecte permettent de disposer d'un nombre croissant d'informations sur les individus et leurs trajectoires, le nombre de dimensions décrivant ces dernières peut lui aussi devenir important. Se posent alors notamment les questions du regroupement des variables en thématiques, de leur pondération et de la sélection des dimensions les plus informatives. Ces problématiques constituent donc autant de prolongements possibles du travail entrepris ici.

Enfin, sur une note plus personnelle, ce stage m'a permis de découvrir l'analyse de séquences, un domaine que je connaissais encore peu, et de travailler sur les trajectoires de vie. Il a également été l'occasion de découvrir de nouvelles méthodes et d'élargir mes connaissances, tant sur le plan statistique qu'informatique. Au-delà de ces apports méthodologiques, le fait de participer à plusieurs réunions du projet ANR TRAVERSÉES ainsi qu'à une conférence m'a permis de découvrir davantage la dimension collective de la recherche, faite d'échanges, de discussions et de comparaison de points de vue, à laquelle mes expériences précédentes m'avaient encore peu exposée. Ce stage a ainsi constitué une expérience particulièrement enrichissante, aussi bien sur le plan scientifique que personnel.

37. *Uniform Manifold Approximation and Projection* (UMAP) est une méthode non linéaire de réduction de dimension qui projette des données de grande dimension dans un espace de plus faible dimension, tout en cherchant à préserver au mieux leur structure locale.

Remerciements

Je tiens à remercier Caroline Bazzoli et Jérôme Gensel de m’avoir prise en stage, de m’avoir accordé un peu de leur précieux temps, d’avoir été là pour guider mon travail durant ces cinq mois et pour leur confiance pour la suite à venir. Je remercie également Anna pour nos échanges et ses conseils. Je voudrais également remercier les membres de l’équipe MESP du CRESI ainsi que les membres de l’équipe Stories pour leur accueil chaleureux. Je souhaite enfin adresser un remerciement particulier à Margaux, Clémence, Julie et Stella, stagiaires à TIMC, avec qui j’ai partagé une grande partie de ces cinq mois, pour les discussions, les pauses et tous les bons moments qui ont rythmé ce stage.

Références

- [1] T. F. Liao, D. Bolano, C. Brzinsky-Fay, B. Cornwell, A. E. Fasang, S. Helske, R. Piccarreta, M. Raab, G. Ritschard, E. Struffolino, and M. Studer, “Sequence analysis: Its past, present, and future,” *Social Science Research*, vol. 107, pp. –, 2022. <https://doi.org/10.1016/j.ssresearch.2022.102772>.
- [2] R. Piccarreta and M. Studer, “Holistic analysis of the life course: Methodological challenges and new perspectives,” *Advances in Life Course Research*, vol. 41, 2019. <https://doi.org/10.1016/j.alcr.2018.10.004>.
- [3] A. Chevaux, “Classification non supervisée de trajectoires de vie : formalisation du modèle salto et étude de mesures de dissimilarité.” 2022.
- [4] J. Gensel, M. Villanova, P. Le Quéau, and D. Noel, “Un modèle multi points de vue pour représenter les trajectoires de vie,” 11 2020.
- [5] A. Gabadinho, G. Ritschard, N. S. Müller, and M. Studer, “Analyzing and visualizing state sequences in r with traminer,” *Journal of Statistical Software*, vol. 40, no. 4, pp. 1–37, 2011. <https://doi.org/10.18637/jss.v040.i04>.
- [6] R. A. Muenchen, “The popularity of data science software.” Visité le 11/08/2026.
- [7] T. Guyet and A. Duvermy, “Towards a library for the analysis of temporal sequences,” 2025.
- [8] E. Zimmermann, M. Budowski, A. Gabadinho, A. Scherpenzeel, R. Tillmann, and B. Wernli, “The swiss household panel,” 12 2009.
- [9] G. Ritschard¹, T. F. Liao, and E. Struffolino, “Strategies for multidomain sequence analysis in social research,” *Sociological Methodology*, vol. 53, no. 2, pp. 288–322, 2023. <https://doi.org/10.1177/00811750231163833>.
- [10] H. Kotthaus, I. Korb, M. Lang, B. Bischl, J. Rahnenführer, and P. Marwedel, “Runtime and memory consumption analyses for machine learning r programs,” *Journal of Statistical Computation and Simulation*, vol. 85, no. 1, pp. 14–29, 2015.
- [11] T. A. Abbott Andrew, “Sequence analysis and optimal matching methods in sociology: Review and prospect,” *Sociological Methods & Research*, vol. 29, no. 1, pp. 3–33, 2000.
- [12] M. Studer, G. Ritschard, A. Gabadinho, and N. S. Müller, “Discrepancy analysis of state sequences,” *Sociological Methods and Research*, vol. 40, no. 3, pp. 471–510, 2011. <https://doi.org/10.1177/0049124111415372>.
- [13] T. F. Liao and A. E. Fasang, “Comparing groups of life-course sequences using the bayesian information criterion and the likelihood-ratio test,” *Sociological Methodology*, vol. 51, no. 1, pp. 44–85, 2021. <https://doi.org/>.
- [14] M. Studer, N. S. Müller, G. Ritschard, and A. Gabadinho, “Classifier, discriminer et visualiser des séquences d’événements,” *Revue des Nouvelles Technologies de l’Information*, vol. Extraction et Gestion des Connaissances, RNTI-E-19, pp. 37–48, 2010.
- [15] X. Wei, Z. Zhang, H. Huang, and Y. Zhou, “An overview on deep clustering,” *Neurocomputing*, vol. 590, 2024. <https://doi.org/10.1016/j.neucom.2024.127761>.

- [16] D. Ienco and R. Interdonato, “Deep multivariate time series embedding clustering via attentive-gated autoencoder,” *Springer Nature Switzerland*, 2020. https://doi.org/10.1007/978-3-030-47426-3_25.
- [17] X. Lin, T. Tian, Z. Wei, and H. Hakonarson, “Clustering of single-cell multi-omics data with a multimodal deep learning method,” *Nature communications*, vol. 590, 2022. <https://doi.org/10.1038/s41467-022-35031-9>.
- [18] N. Barban and F. Billari, “Classifying life course trajectories: a comparison of latent class and sequence analysis,” *Journal of the Royal Statistical Society*, vol. 61, pp. 765–784, 2012. <https://doi.org/10.1111/j.1467-9876.2012.01047.x>.
- [19] G. Ritschard, “Measuring the nature of individual sequences,” *Sociological Methods Research*, 2021.
- [20] Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, 1987.
- [21] X. V. Nguyen, J. Epps, and J. Bailey, “Information theoretic measures for clusterings comparison: is a correction for chance necessary?,” *Proceedings of the 26th International Conference on Machine Learning*, pp. 1073–1080, 2009. <https://doi.org/10.1145/1553374.1553511>.
- [22] H. Caliński, “A dendrite method for cluster analysis,” *Communications in Statistics*, 1974.
- [23] L. Hubert, “A general statistical framework for assessing categorical clustering in free recall,” *Psychological Bulletin*, 1976.
- [24] U. G. Alpes, “Plan de mobilité durable 2025,” 2025. <https://www.univ-grenoble-alpes.fr/universite/ambition-et-strategie/documents-strategiques/transformation-ecologique/plan-de-mobilite-durable-2025-1696922.kjsp>.
- [25] HP2, “Centre de recherche en santé intégrée (cresi),” 2026. <https://hp2.univ-grenoble-alpes.fr/fr/centre-recherche-en-sante-integree-cresi>.
- [26] D. Larousserie, “La recherche au défi de la sobriété énergétique du numérique - journal le monde,” 2023. https://www.lemonde.fr/sciences/article/2023/05/01/la-recherche-au-defi-de-la-sobriete-energetique-du-numerique_6171677_1650684.html.
- [27] Entrelacs, “75 pourquoi faut-il demanteler le numérique?,” 2025. <https://entrelacs.org/podcast/pouquoi-demanteler-le-numerique/>.
- [28] U. G. Alpes, “Plan d’action égalité femmes hommes 2025-2028,” 2025. <https://www.univ-grenoble-alpes.fr/universite/ambition-et-strategie/documents-strategiques/egalite-et-lutte-contre-les-discriminations/plan-d-action-egalite-femmes-hommes-2025-2028-1691221.kjsp>.
- [29] A. Chemin, “Le racisme à l’université, un impensé de la recherche - journal le monde,” 2026. https://www.lemonde.fr/idees/article/2026/02/13/le-racisme-a-l-universite-un-impense-de-la-recherche_6666657_3232.html.
- [30] M. I. des Sciences sociales et des Humanités d’Alsace, “Inégalités dans et en recherche - colloque,” 2025. <https://www.misha.fr/agenda/colloquemisha2025-inegalites/>.
- [31] L. Unterlerchner and M. Studer, “What are we looking for? a comparative review of clustering algorithms and cluster quality indices for sequence analysis,” *LIVES Working Papers*, pp. 1–74, 2026. <https://doi.org/10.12682/lives.2296-1658.2026.108>.

7 Annexes

Critère d'évaluation	TraMineR	TanaT	sequenzo
Formats d'entrée supportés	STS, SPS, SPELL, DSS, SRS, TSE, SS (+ FCE, HSPELL avec TraMineRextras)	SPELL (sans précision des dates de fin)	STS, LONG
Formats pour l'objet séquence	STS, SPS, SPELL	SPELL (sans précision des dates de fin)	STS
Outils de conversion	Fonctions natives intégrées	Prétraitement externe requis	Fonction native intégrée de STS à LONG
Prise en compte des poids de redressement pour les analyses ultérieures	Supportée nativement	Non supportée	Supportée nativement

TABLE 1 – Comparaison des formats de données acceptés, des outils de conversion et la prise en charge des poids de redressement pour chaque librairie

STS : une ligne par objet et une colonne par point dans le temps ; **SPS** : une liste de couples (type, durée) par objet ; **SPELL** : une colonne d'identifiant, une date de début, une date de fin et une colonne d'état ; **DSS** : liste ordonnée des états distincts ; **SRS** : séquences répliquées décalées ; **TSE** : une colonne d'identifiant, une colonne indiquant le point de temps de chaque transition et une colonne précisant le type de transition (ex. a>b) ; **SS** : une chaîne de caractères contenant les états à chaque point de temps (ex. a-a-a-b-b-c-c) ; **FCE** : une colonne d'identifiant et une colonne par état de l'alphabet indiquant le point de temps de sa première occurrence ; **HSPELL** : une ligne par objet et autant de triplets de colonnes début-fin-état que le nombre maximal d'épisodes dans le jeu de données ; **LONG** : une ligne par individu par point de temps. Tous les formats ne contiennent pas le même niveau de détail. Des exemples ces formats sont proposés dans l'annexe pour la librairie TraMineR.

Nom	Thématique	Type	Format	Nb. obj.	Long.	Nb. états	Nb. cov.
actcal	Statut d'emploi (SHP)	Évè. ou états	STS / TSE	2 000	12	4	11
biofam	Statut familial (SHP)	États	STS	2 000	16	8	7
mvad	Transition école-travail	États	STS	712	72	6	13
ex1	Pédagogique (manquants)	États	STS	7	Var.	4	-
ex2	Pédagogique (poids)	États	SS	100/6	Var.	4	-
famform	Historique familial	États	SS	5	Var.	5	-

TABLE 2 – Jeux de données d'exemple fournis par le package TraMineR

Nom	Thématique	Type	Format	Nb. obj.	Long.	Nb. états	Nb. cov.
MIMIC-IV (adm)	Registres médicaux	Interv./Év.	SPELL	100	Var.	9 ou 10	2
MIMIC-IV (pro)	Registres médicaux	Interv./Év.	SPELL	92	Var.	2 ou 352	2
MOOC-events	Utilisation MOOC	Évènements	SPELL	130	Var.	12 ou 30	-

TABLE 3 – Jeux de données d'exemple fournis par la bibliothèque TanaT

Nom	Thématique	Type	Nb. obj.	Long.	Nb. états	Nb. cov.
biofam + version binaire pour les états (left, married et child)	Statut familial (Swiss Household Panel)	États	2000	16	8	7
Chinese Territories' Colonial History	Statut de souveraineté de territoires chinois	États	11	111	7	-
Country child mortality	Mortalité infantile dans les pays du monde	États	194	302	10	-
Country child mortality	Produit intérieur brut par habitant	États	193	223	5	-
Country life expectancy	Espérance de vie dans les pays du monde	États	194	302	10	-
country CO ₂ emissions (découpage en déciles ou quintiles / découpage local ou global) ³⁸	Emissions de CO ₂	États	193	223	5 ou 10	-
dyadic (children)	Formation de la famille de 15 à 39 ans, enfants	États	60	24	6	1
dyadic (parents)	Formation de la famille de 15 à 39 ans, parents	États	60	24	6	1
mvad	Transition école-travail	États	712	72	6	13
pairfam activity (par mois)	Trajectoires d'emploi / d'activité	États	1027	264	8	16
pairfam activity (par années)	Trajectoires d'emploi / d'activité	États	1027	22	8	-
pairfam family (par mois)	Formation de la famille	États	1027	264	9	16
pairfam family (par années)	Formation de la famille	États	1027	22	9	-
political science aid shock	Sciences politiques	États	165	36	2	-
political science donor fragmentation	Sciences politiques	États	156	36	3	-
College Students' stress	Évolution du stress d'étudiants	États	76	8	3	5

TABLE 4 – Jeux de données d'exemple fournis par la bibliothèque **sequenzo**

Listing 1 – Implémentation proposée pour la génération de séquences avec **TraMineR**

```

1 # Exemple d'utilisation de la fonction seqgen.
2 seqgen(n=3, length=8, alphabet = 1:3, p=c(0.2,0.1,0.7))

```

38. Voir la documentation pour détails : <https://sequenzo.yuqi-liang.tech/en/datasets/CO2-emissions>.

Listing 2 – Implémentation proposée pour la génération de séquences avec TanaT

```

1  # Sequences d'etats
2  state_data = generate_state_sequences(n_seq= 10, seq_size=[4,5,3], vocabulary=[1,2,3],
    missing_data=0.1, entity_feature="state_type", seed=32)
3
4  # Sequences d'evenements
5  event_data = generate_event_sequences(n_seq= 10, seq_size=[4,5,3], vocabulary=[1,2,3],
    missing_data=0, entity_feature="event_type", seed=32)
6
7  # Sequences d'intervalles
8  interval_data = generate_state_sequences(n_seq= 10, seq_size=[4,5,3], vocabulary=[1,2,3],
    missing_data=0.2, entity_feature="interval_type", seed=32)

```

Listing 3 – Implémentation proposée pour la génération de séquences avec sequenzo

```

1  import numpy as np
2  import pandas as pd
3
4  # Definition d'une fonction
5  def seqgen(
6      n: int, length, alphabet: list, p: list = None, id_prefix: str = "seq", missing: str = None,
7      p_missing: float = 0.0, pad: str = "NA", seed: int = None
8  ) -> pd.DataFrame:
9      """
10
11      Genere n sequences aleatoires au format STS (State Sequence).
12
13      Parametres
14      -----
15      n : int
16          Nombre de sequences a generer.
17      length : int ou array-like de longueur n
18          Longueur de chaque sequence. Si un entier est fourni, toutes les
19          sequences ont la meme longueur. Si un vecteur est fourni, chaque
20          sequence i a la longueur length[i].
21      alphabet : list
22          Liste des etats possibles.
23      p : list, optional
24          Probabilites associees a chaque etat (doit sommer a 1).
25          Si None, distribution uniforme.
26      id_prefix : str, optional
27          Prefixe pour les identifiants de sequences (default : "seq").
28      missing : str, optional
29          Valeur representant les donnees manquantes (ex : "*").
30          Si None, pas de donnees manquantes generees.
31      p_missing : float, optional
32          Probabilite qu'un etat soit manquant (entre 0 et 1). Ignore si missing=None.
33      pad : str, optional
34          Valeur utilisee pour remplir les positions inexistantes des sequences
35          plus courtes que la longueur maximale (default : "NA").
36      seed : int, optional
37          Graine pour la reproductibilite.
38
39      Retourne
40      -----
41      pd.DataFrame
42          DataFrame au format STS : colonne 'id' + colonnes 't1' a 't{max_length*}'.

```



```

41     Les positions hors longueur individuelle sont remplies avec `pad`.
42     """
43
44     # Reproductibilite
45     rng = np.random.default_rng(seed)
46
47     # Formatage de length en vecteur
48     if np.isscalar(length):
49         lengths = np.full(n, length, dtype=int)
50     else:
51         lengths = np.asarray(length, dtype=int)
52         if len(lengths) != n:
53             raise ValueError("'length' doit etre un scalaire ou un vecteur de longueur n.")
54         if np.any(lengths <= 0):
55             raise ValueError("Toutes les valeurs de 'length' doivent etre strictement positives.")
56
57     max_length = lengths.max()
58
59     # Validation des probabilites
60     if p is not None:
61         p = np.array(p, dtype=float)
62         if len(p) != len(alphabet):
63             raise ValueError("'p' doit avoir la meme longueur que 'alphabet'.")
64         if not np.isclose(p.sum(), 1.0):
65             raise ValueError("Les probabilites dans 'p' doivent sommer a 1.")
66     else:
67         p = np.ones(len(alphabet)) / len(alphabet)
68
69     if not (0.0 <= p_missing < 1.0):
70         raise ValueError("'p_missing' doit etre compris entre 0 (inclus) et 1 (exclus).")
71
72     # Generation sequence par sequence (longueurs variables)
73     sequences = []
74     for i in range(n):
75         seq = rng.choice(alphabet, size=lengths[i], p=p).astype(object)
76
77         # Injection de valeurs manquantes
78         if missing is not None and p_missing > 0.0:
79             mask = rng.random(size=lengths[i]) < p_missing
80             seq[mask] = missing
81
82         # Padding jusqu'a max_length
83         padding = np.full(max_length - lengths[i], pad, dtype=object)
84         sequences.append(np.concatenate([seq, padding]))
85
86     # Construction du dataframe au format STS
87     ids = [f"{id_prefix}_{i+1}" for i in range(n)]
88     col_names = [f"t{j+1}" for j in range(max_length)]
89     df = pd.DataFrame(sequences, columns=col_names)
90     df.insert(0, "id", ids)
91
92     return df
93
94     # Exemple de 10 sequences de longueur 12 avec pour alphabet {a, b, c} avec une repartition
95     p=[0.75,0.1,0.15] et 10% de donnees manquantes
96     seqgen(n=3, length=[8,10,7], alphabet=["a","b","c"], p=[0.75,0.1,0.15], missing="Missing",
97           p_missing=0.1)

```

Ecosystème	TraMineR		TanaT		sequenzo	
Fonctionnalité	Données brutes	Visualisation	Données brutes	Visualisation	Données brutes	Visualisation
Alphabet	✓		✓		✓	
Visualisation de séquences		✓		✓		✓
Fréquence des séquences	✓	✓	–	–	–	✓
Fréquence des états co-occurents	✓	–	–	–	–	–
Séquence représentative	✓	✓	–	–	–	✓
Fréquence des états	✓	✓	–	✓	–	✓
Table des transitions	✓	–	–	–	–	✓
Distribution transversale	✓	✓	–	✓	–	✓
Entropie <i>transversale</i>	–	✓	–	–	✓	✓
Single medoid plot	–	–	–	–	–	~
Relative frequency plot	–		–	–	–	✓

TABLE 5 – Comparaison des statistiques descriptives disponibles à l’échelle de l’ensemble des séquences et des visualisations associées (✓ : disponible ; ~ : documenté mais non fonctionnel dans l’environnement testé ; – : absent ; grisé : non pertinent).

Listing 4 – Implémentation calcul de la fréquence des séquences pour TanaT

```

1 # Extraction des sequences au format SS (a-a-b-b-c-...)
2 biofam_expanded = biofam_expanded.sort_values(by=['id', 'timestep'])
3 full_sequences = biofam_expanded.groupby('id')['states'].apply(lambda x: '-'.join(x))
4 # Comptage et tableau resultats
5 seq_freq = full_sequences.value_counts(normalize=True)

```

Listing 5 – Implémentation calcul de la fréquence des séquences pour sequenzo

```

1 # Comptage des sequences
2 arr = biofam[timepoints_columns].values #array comprenant les valeurs des sequences
3 sequences_tuples = [tuple(row) for row in arr] #transforme les sequences sous forme de tuple
4 from collections import Counter
5 counts = Counter(sequences_tuples) #compte le nombre de sequences
6 # Tableau resultats
7 df_freq = pd.DataFrame([
8     {"sequence": list(seq), "frequency": freq/len(arr)}
9     for seq, freq in counts.most_common()
10 ])

```

Listing 6 – Exemple d’implémentation d’un chronogramme selon la variable catégorielle **sex** dans TanaT

```

1 # Separation de biofam en 2 selon le sexe (variable sex)
2 criterion_man = StaticCriterion(query="sex == 'man'")
3 criterion_woman = StaticCriterion(query="sex == 'woman'")
4 man_seqpool = biofam_pool.filter(criterion_man)
5 woman_seqpool = biofam_pool.filter(criterion_woman)
6

```

```

7 # Visualisation
8 SequenceVisualizer.timeline(stacking_mode="flat") \
9     .title("Biofam dataset - Femmes") \
10    .colors(biofam_colors) \
11    .marker(spacing=0.9) \
12    .legend(show=True, title="Etats", loc="center left") \
13    .x_axis(label="Temps") \
14    .draw(woman_seqpool)
15
16 SequenceVisualizer.timeline(stacking_mode="flat") \
17     .title("Biofam dataset - Hommes") \
18     .colors(biofam_colors) \
19     .marker(spacing=0.9) \
20     .legend(show=True, title="Etats", loc="center left") \
21     .x_axis(label="Temps") \
22     .draw(man_seqpool)

```

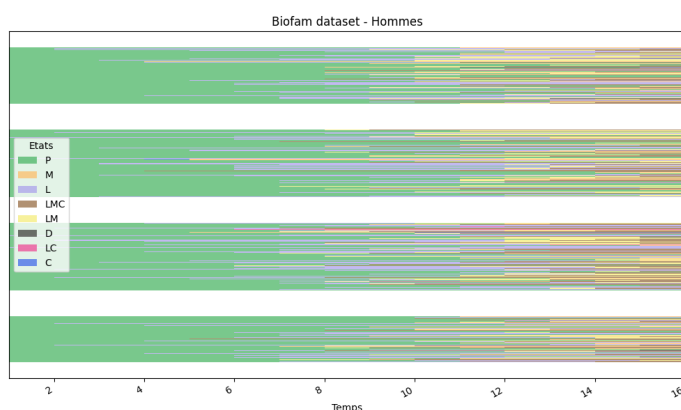


FIGURE 8 – Sortie TanaT pour les hommes (*man*)

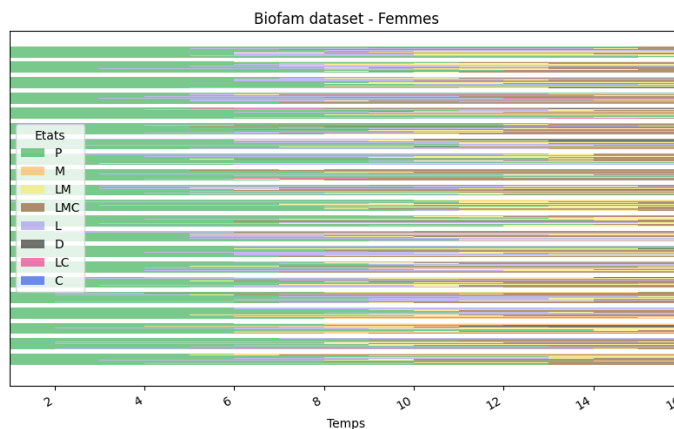


FIGURE 9 – Sortie TanaT pour les femmes (*woman*)

FIGURE 10 – Chronogrammes avec TanaT selon la variable **sex**

Listing 7 – Exemple d'implémentation d'un chronogramme selon la variable catégorielle **sex** dans **sequenzo**

```

1 sequenzo.plot_sequence_index(biofam_seq, group_by_column="sex")

```

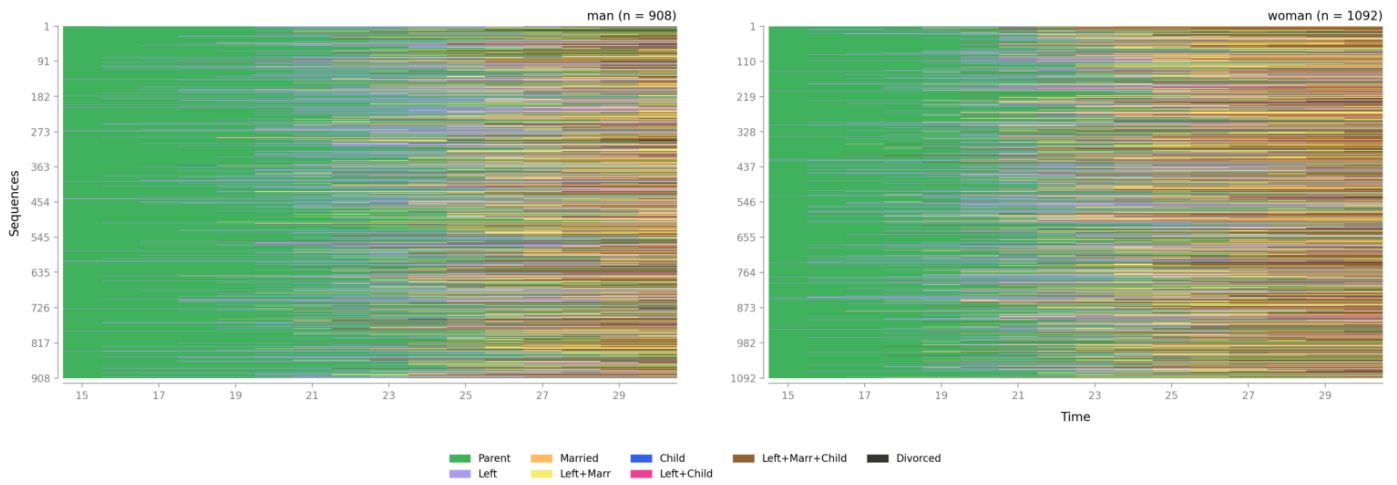


FIGURE 11 – Chronogrammes avec `sequenozo` selon la variable `sex`

Listing 8 – Exemple d’implémentation d’un chronogramme selon la variable catégorielle `sex` dans `TraMineR`

```
1 seqIplot(biofam.seq, group=biofam$sex, with.legend = "right", cex.axis=0.8, cex.legend=1)
```

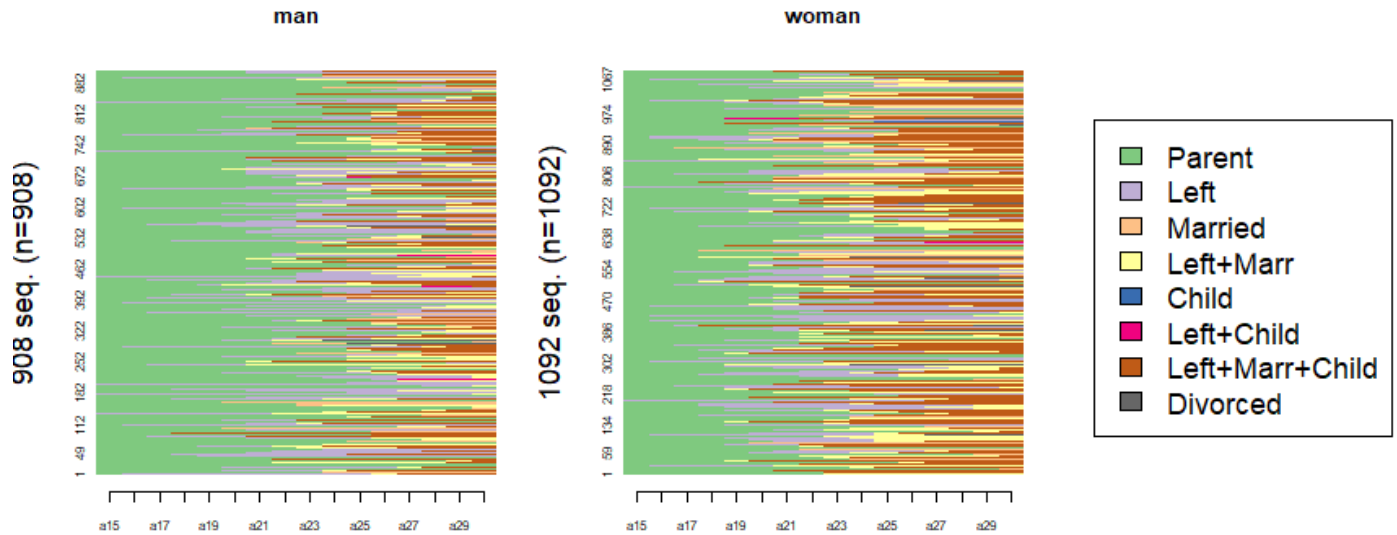


FIGURE 12 – Chronogrammes avec `TraMineR` selon la variable `sex`

Statistique	TraMineR	sequenzo	TanaT
Nombre de sous-séquences	✓	✓	–
Longueur de la séquence	✓	–	✓
Liste des états distincts	✓	–	–
Nombre d'états distincts	–	–	✓
Fréquence des états	–	✓	–
Nombre de transitions	–	✓	✓
Entropie (<i>within</i>)	✓	✓	✓
Complexité	✓	✓	–
Turbulence	✓	✓	–
Volatilité	✓	✓	–
Post-Negative	✓	~	–
Integration	✓	~	–
Badness	✓	~	–
Degradation	✓	~	–
Precarité	✓	~	–
Insecurité	✓	~	–
Variance de la durée d'un épisode	–	✓	–
Durée moyenne d'un épisode	–	–	✓
Durée médiane d'un épisode	–	–	✓
Écart-type de la durée d'épisode	–	–	✓
Visualisation	–	✓	–

TABLE 6 – Statistiques descriptives portant sur les séquences individuelles (✓ : présent ; ~ : documenté mais non fonctionnel dans l'environnement testé ; – : absent)

Distance	Famille	sequenzo	TraMineR	TanaT
OM	Édition	✓	✓	✓
OMloc	Édition	✓	✓	–
OMspell	Édition	✓	✓	–
OMslen	Édition	✓	✓	–
OMspellNew	Édition	✓	–	–
OMtspell	Édition	✓	–	–
OMstran	Édition	✓	✓	–
TWED	Édition	✓	✓	–
HAM	Appariement positionnel	✓	✓	–
DHD	Appariement positionnel	✓	✓	–
LCS	Correspondance de sous-séquence	✓	✓	✓
NMS	Correspondance de sous-séquence	✓	✓	–
NMSMST	Correspondance de sous-séquence	✓	✓	–
SVRspell	Correspondance de sous-séquence	✓	✓	–
LCP	Préfixe commun	✓	✓	✓
RLCP	Préfixe commun	✓	✓	–
LCPspell	Préfixe commun	✓	–	–
RLCPspell	Préfixe commun	✓	–	–
LCPmst	Préfixe commun	✓	–	–
RLCPmst	Préfixe commun	✓	–	–
LCPprod	Préfixe commun	✓	–	–
RLCPprod	Préfixe commun	✓	–	–
DTW	Warping	–	–	✓
Soft-DTW	Warping	–	–	✓
χ^2	Distribution	✓	✓	✓
Euclid	Distribution	✓	✓	–
Linear Pairwise	Distribution	–	–	✓

TABLE 7 – Distances disponibles pour chaque librairie classées par famille (✓ : présent ; – : absence)

OM à **OMstran**: Optimal Matching et variations. **TWED**: Time Warp Edit Distance. **HAM**: Hamming Distance. **DHD**: Dynamic Hamming Distance. **LCS**: Longest Common Subsequence. **NMS**: Number of Matching Subsequences. **SVRspell**: Spell-based Vector Representation. **LCP**: Longest Common Prefix. **RLCP**: Reversed LCP. **DTW**: Dynamic Time Warping. **Soft-DTW**: Soft Dynamic Time Warping. χ^2 : Temps passé dans chaque état. **Euclid**: Distance euclidienne $d(x, y) = \sum_{t=1}^L (x_t - y_t)^2$. **Linear Pairwise**: Comparaison état à état $d(x, y) = \sum_{t=1}^L d(x_t, y_t)$.

Tâche	TraMineR (s)	TanaT (s)	sequenzo (s)
Calcul distance OM	0.215	1.893	0.257
Clustering hiérarchique sur 1 variable	0.199	0.106	0.175
Clustering EA	1.841	1.27	3.30
Clustering CAT	11.074	1.70	462.722
Clustering DAT	2.057	15	1.447
Clustering CombT	4.525	5.10	13.175

TABLE 8 – Comparaison des performances sur les tâches de clustering (en secondes) pour chaque librairie

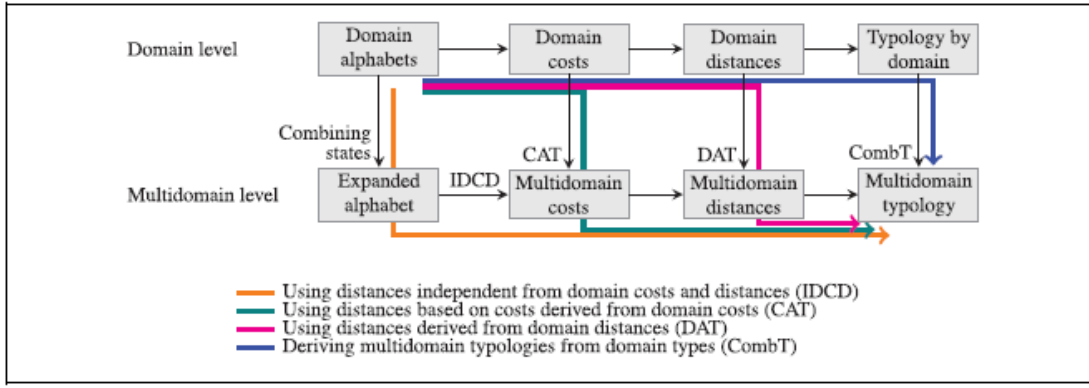


FIGURE 13 – Illustration des quatre approches pour les trajectoires multidimensionnelles réalisée par *Ritschard et al., 2023* [9].

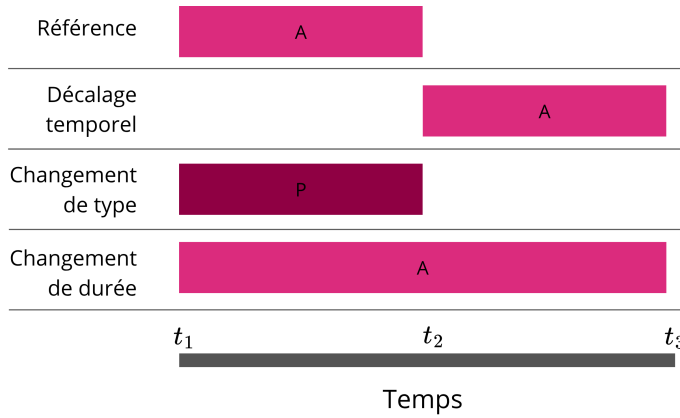


FIGURE 14 – Séquence d'intervalles de référence et séquences d'intervalles présentant un décalage temporel, un changement de type ou un changement de durée.

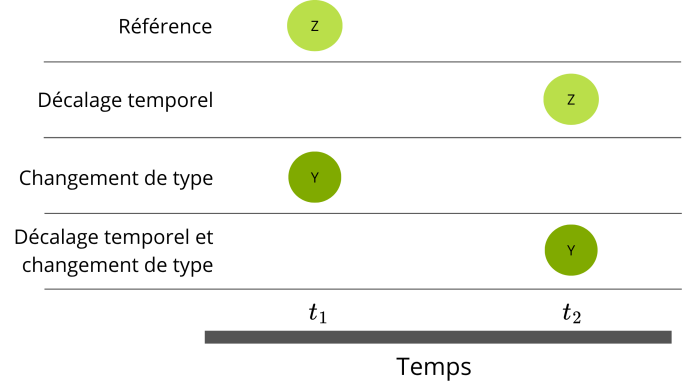


FIGURE 15 – Séquence d'intervalles de référence et séquences d'intervalles présentant un décalage temporel, un changement de type ou les deux.

Séquence comparée à la référence	Chi2	DTW	Soft-DTW	Linear Pairwise	OM	LCP	LCS
Référence	0	0	0	0	0	1	1
Décalage temporel	0	0	0	0	0	1	1
Changement de type	6.93	1	1	1	1	0	0
Changement de durée	2.83	0	0	0	0	1	1

TABLE 9 – Distances entre la séquence d'intervalles de référence et d'autres séquences d'intervalles présentant un décalage temporel, un changement de type ou un changement de durée.

Séquence comparée à la référence	Chi2	DTW	Soft-DTW	Linear Pairwise	OM	LCP	LCS
Référence	0	0	0	0	0	1	1
Décalage temporel	0	0	0	0	0	1	1
Changement de type	1.41	1	1	1	1	0	0
Décalage temporel et changement de type	1.41	1	1	1	1	0	0

TABLE 10 – Distances entre la séquence d'événements de référence et d'autres séquences d'événements présentant un décalage temporel, un changement de type ou les deux.

FIGURE 16 – Schéma de l'effet d'une perturbation par repoussement

FIGURE 17 – Schéma de l'effet d'une perturbation par mutation.

FIGURE 18 – Schéma de l'effet d'une perturbation par troncature

